

大模型入门手册

WPC

2026 年 6 月 25 日

目录

第一部分 深度学习基础	7
第一章 感知机与神经网络	9
1.1 感知机简介	9
1.2 感知机能力与局限	10
1.3 神经网络简介	12
1.4 从数据中学习	13
第二章 误差反向传播	15
2.1 计算图简介	15
2.2 反向传播	16
2.3 参数优化方法	17
2.4 归一化和正则化	20
第三章 CNN	23
3.1 CNN 简介	23
3.2 卷积层	23
3.3 池化层	25
3.4 CNN 的发展	26
第二部分 自然语言处理	27
第四章 单词表示与分词	29
4.1 单词表示	29
4.2 分词	31
第五章 语言模型	33
5.1 语言模型简介	33
5.2 RNN	34
5.3 LSTM 和 GRU	35

第六章 Transformer	39
6.1 Seq2Seq 模型	39
6.2 Self-Attention	39
6.3 位置编码	42
6.4 编码器与解码器	43
第七章 注意力机制优化	47
7.1 GQA	47
7.2 FlashAttention	47
7.3 PagedAttention	48
7.4 线性注意力	49
第三部分 大语言模型应用	51
从 Transformer 到智能体	53
第八章 提示词工程	55
8.1 什么是提示词工程	55
8.2 零样本提示	55
8.3 少样本提示	55
8.4 思维链提示	56
8.5 提示词工程的实践技巧	56
8.6 提示词工程的本质	57
第九章 检索增强生成 (RAG)	59
9.1 大语言模型的局限性	59
9.2 什么是 RAG	59
9.3 文本嵌入	60
9.4 向量数据库	60
9.5 RAG 的工作流程	61
9.6 RAG 的优势	62
9.7 RAG 的挑战	62
第十章 智能体与工具使用	63
10.1 从对话到行动	63
10.2 什么是智能体	63
10.3 Agent 循环：智能体的心跳	63
10.4 工具系统	66
10.5 函数调用：连接模型与工具的桥梁	70

目录5

10.6 系统提示词：智能体的灵魂

71

10.7 子智能体：分工协作

73

10.8 上下文管理

74

10.9 记忆系统

75

10.10 编码智能体：Claude Code

76

10.11 智能体的挑战与展望

78

第一部分

深度学习基础

第一章 感知机与神经网络

1.1 感知机简介

感知机 (Perceptron) 由美国学者 Frank Rosenblatt 于 1957 年提出，作为神经网络最早期的算法之一，感知机奠定了深度学习发展的基础，至今仍具有重要的学习和研究价值。

在感知机之前，逻辑运算和模式识别主要依赖人工设计的规则，而感知机提供了一种自动学习的方法：通过调整权重参数，使机器能够从训练数据中提取规律，从而对新的输入做出正确的判断。这种“从数据中学习”的思想，正是现代机器学习的核心。

图 1.1 是一个接收两个输入信号的感知机的例子。感知机接收多个输入信号，并输出一个信号，在感知机模型中，输入与输出的信号通常是二值信号 (0/1)。 x_1 、 x_2 是输入信号， y 是输出信号， w_1 、 w_2 是对应权重。图中的 $\bigcirc$ 代表“神经元”，输入信号被送往神经元时，会被分别乘以固定的权重，神经元会计算传送过来的信号的总和，当这个总和超过了某个阈值 θ ，神经元输出 1，这也被称为“神经元被激活”。

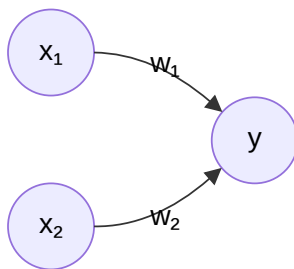


图 1.1: 感知机示例

把上述内容用数学式来表示。

$$y = \begin{cases} 0, & w_1x_1 + w_2x_2 \leq \theta \\ 1, & w_1x_1 + w_2x_2 > \theta \end{cases}$$

1.2 感知机能力与局限

从上面的公式可以看出，感知机的本质是通过权重和阈值对输入信号进行分类。那么，感知机能否用来实现基本的逻辑运算呢？答案是肯定的。下面考虑用感知机来实现与门，表 1.1 是与门的真值表。

x_1	x_2	$y = x_1 \wedge x_2$
0	0	0
0	1	0
1	0	0
1	1	1

表 1.1: 与门的真值表

我们需要做的就是确定能满足真值表的参数的值，也就是找到 w_1 、 w_2 、 θ ，使得仅当 x_1 和 x_2 同时为 1 时，信号的加权总和才会超过给定的阈值 θ 。实际上，参数的选择方法有无数多种。比如，当 $(w_1, w_2, \theta) = (0.5, 0.5, 0.7)$ 时，可以满足条件；此外，当 $(w_1, w_2, \theta) = (0.5, 0.5, 0.8)$ 或者 $(1.0, 1.0, 1.0)$ 时，同样也满足与门的条件。

为了方便后续讨论，我们把上面感知机实现的数学式子改写一下，用偏置 b 代替阈值 θ ($b = -\theta$)。

$$y = \begin{cases} 0, & b + w_1x_1 + w_2x_2 \leq 0 \\ 1, & b + w_1x_1 + w_2x_2 > 0 \end{cases}$$

前面我们用感知机实现了与门，现在，我们来探讨一个更具挑战性的例子——异或门。表 1.2 是异或门的真值表，应该设定什么样的参数能满足这个真值表呢？

x_1	x_2	$y = x_1 \oplus x_2$
0	0	0
0	1	1
1	0	1
1	1	0

表 1.2: 异或门的真值表

实际上，用前面介绍的感知机是无法实现这个异或门的。这是因为感知机的本质是生成一条直线来划分空间，以我们之前实现的与门为例，生成的直线为 $-0.8 + 0.5x_1 + 0.5x_2 = 0$ 。

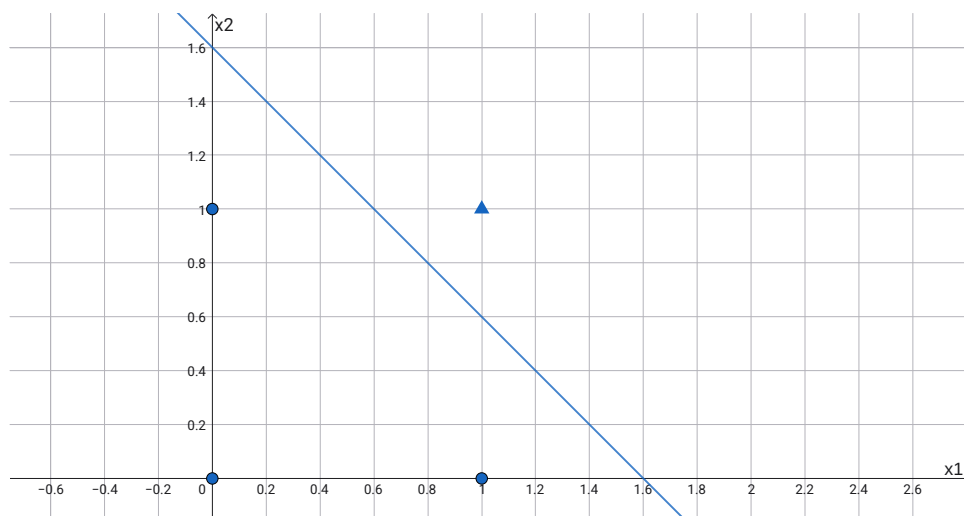


图 1.2: 与门可视化

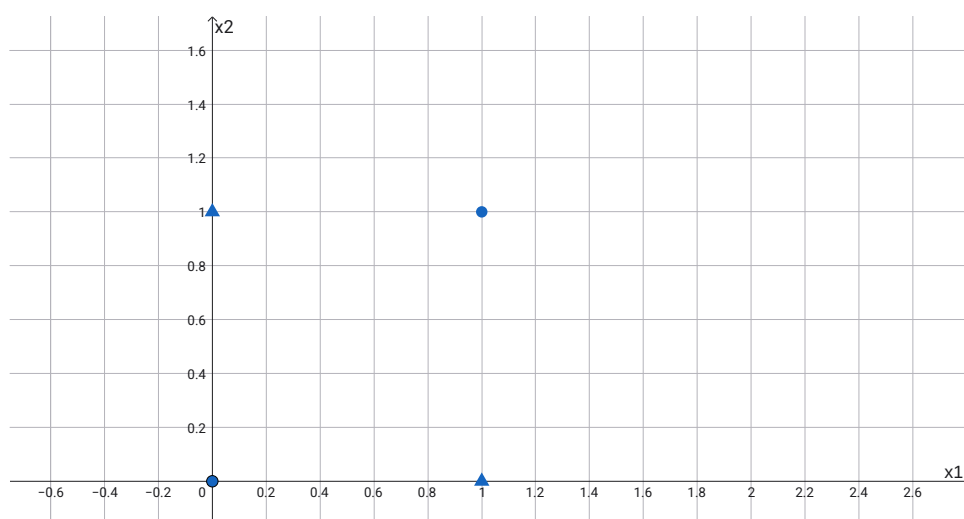


图 1.3: 异或门可视化

想要用一条直线将图 1.3 中的数据分开，是做不到的。感知机只能表示线性可分的空间，因此感知机无法实现异或门。

那么，有没有办法绕过这个限制呢？前面我们已经用感知机实现了与门，类似地，与非门和或门也可以用感知机来实现。在数字逻辑中，通过与门、与非门、或门的组合，可以构建出异或门。既然这些基本门电路都能用感知机表示，那么我们自然可以通过“多层叠加”感知机来处理非线性可分的问题。

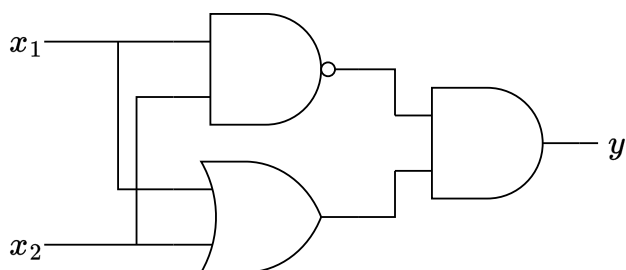


图 1.4: 通过组合实现异或门

1.3 神经网络简介

在前面的感知机模型中，参数的确定并不是由计算机自动完成的，而是由我们人为地根据真值表等“训练数据”手动计算得出。为了让计算机能够自动学习和调整参数，神经网络的概念应运而生。

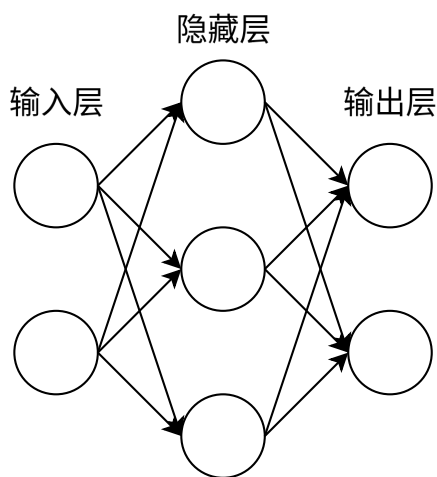


图 1.5: 神经网络示例

将之前感知机的函数改写成以下更简洁的形式。

$$y = h(b + w_1x_1 + w_2x_2) \quad h(x) = \begin{cases} 0, & x \leq 0 \\ 1, & x > 0 \end{cases}$$

$h(x)$ 函数会将输入信号的总和转换为输出信号，被称为激活函数（activation function）。我们之前使用的激活函数是“阶跃函数”，一旦输入超过阈值，就切换输出。除了这个外，神经网络中常用的激活函数有 sigmoid 函数、ReLU（Rectified Linear Unit）函数。

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad \text{ReLU}(x) = \begin{cases} 0, & x \leq 0 \\ x, & x > 0 \end{cases}$$

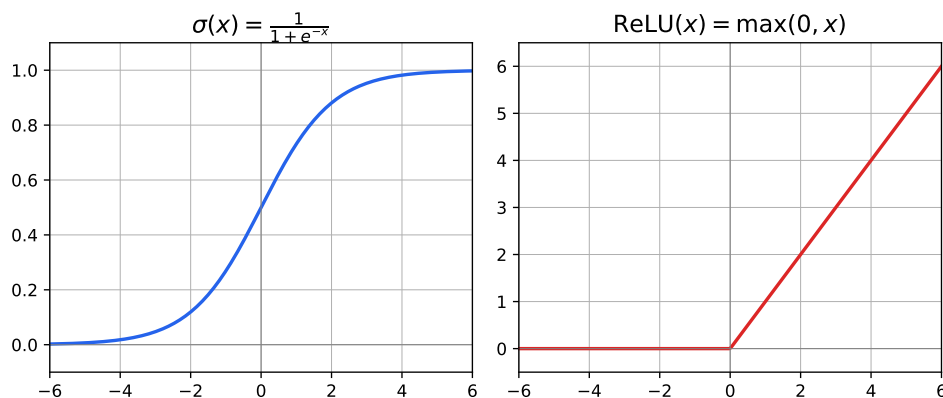


图 1.6: Sigmoid 函数与 ReLU 函数

对于输出层，根据任务的不同，我们可能需要选择不同的激活函数。例如，在二分类问题中，输出层可以直接使用 sigmoid 函数，将输出转换为 0 到 1 之间的概率值。而在多分类问题中，我们通常需要输出每个类别的概率分布，这时就需要使用 softmax 函数对输出层进行处理。

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

1.4 从数据中学习

神经网络“从数据中学习”，由数据自动决定权重参数的值。为了达成这个目的，我们先要决定损失函数（loss function），之后，神经网络就以这个指标为基准，寻找最优权重参数。常用的损失函数有均方误差（mean squared error）和交叉熵误差（cross entropy error）。

$$\text{MSE} = \frac{1}{2} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad \text{CE} = - \sum_{i=1}^K \hat{y}_i \log(y_i)$$

这里， y_i 是表示神经网络的输出， $\hat{y}_i$ 表示监督数据， i 表示数据的维数。交叉熵误差主要用于多分类问题，实际上他只计算对应正确解标签的输出，假设正确解标签的索引是“1”，与之对应的神经网络的输出是 0.7，则交叉熵误差是 $-\log 0.7$ 。

使用数据进行学习，就是针对训练数据计算损失函数的值，找出使该值尽可能小的参数。当训练数据量很大（例如几百万、几千万条）时，每次都使用全部数据来计算损失函数是不现实的。因此，我们从训练数据中随机抽取一小批数据（称为 mini-batch），仅用这批数据来近似计算损失函数并更新参数。例如，从 60000 条训练数据中随机抽取 100 条，用这 100 条数据计算损失并更新参数，然后重新抽取下一批 100 条，重复这一过程。这种学习方式称为 mini-batch 学习。

学习的目标是最小化损失函数，梯度法是实现这一目标的常用方法。它的核心思想是，计算损失函数关于每个参数的梯度，然后沿着梯度的反方向更新参数。因为梯度指向函数值增长最快的方向，反方向就是下降最快的方向，反复沿着这个方向更新，就能逐步逼近最小值。具体来说，对于参数 x_1, x_2 ，梯度法的更新规则如下：

$$\begin{aligned}x_1^{(t+1)} &= x_1^{(t)} - \eta \frac{\partial f}{\partial x_1}(x_1^{(t)}, x_2^{(t)}) \\x_2^{(t+1)} &= x_2^{(t)} - \eta \frac{\partial f}{\partial x_2}(x_1^{(t)}, x_2^{(t)})\end{aligned}$$

η 被称为学习率 (learning rate)。

为了计算梯度，我们可以使用数值微分 (Numerical Differentiation) 来近似求导：给输入一个微小的扰动 h (例如 $h = 10^{-4}$)，然后通过计算函数值的变化率来近似导数：

$$\frac{d}{dx}f(x) \approx \frac{f(x+h) - f(x)}{h} \quad (h \rightarrow 0)$$

这种方法不需要推导解析表达式，可以直接通过代码计算任意函数的导数。

第二章 误差反向传播

在上一章中，我们提到可以通过梯度法来更新参数、最小化损失函数，并使用数值微分来近似计算梯度，这种方法虽然简单直观，但当网络层数增多、参数量变大时，数值微分需要对每个参数分别施加微小扰动并重新计算整个网络，计算量会急剧增大，效率极低。为了解决这个问题，我们需要一种更高效的求导方法——误差反向传播。

2.1 计算图简介

计算图是理解反向传播的关键，计算图（Computational Graph）是一种将复杂的数学表达式分解为基本运算单元并用图结构表示的方法。在计算图中，节点（Node）表示变量或操作（如加法、乘法、激活函数等），边（Edge）表示数据在操作之间的流动或依赖关系。

用计算图解题的情况下，需要先构建计算图，然后在计算图上，从左向右进行计算。这里的“从左向右进行计算”被称为正向传播（forward propagation）。事实上，也存在反向传播（backward propagation），反向传播将在接下来的导数计算中发挥重要作用。

假设小明在超市买了两个 5 元/个的苹果，消费税为 10%，请计算他需要支付的总金额。这个问题可以用图 2.1 来表示，从左向右逐步计算：先算 $5 \times 2 = 10$ （苹果总价），再算 $10 \times 1.1 = 11$ （含税价格），最终得到小明需要支付 11 元。

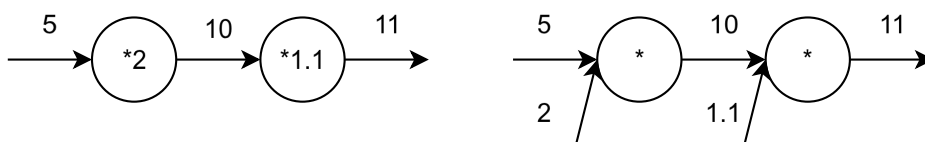


图 2.1: 计算图示例

计算图的特征是可以通过传递“局部计算”获得最终结果：每个节点只负责自己简单的运算，不需要关心整个计算过程的全貌。这就带来一个优点：无论全局是多么复杂的计算，都可以通过局部计算使各个节点致力于简单的计算，从而简化问题。

2.2 反向传播

再来思考一下之前的问题，我们计算了购买 2 个苹果时加上消费税最终需要支付的金额。然后现在我们想知道苹果价格的上涨会在多大程度上影响最终的支付金额，即求“支付金额关于苹果的价格的导数”。具体来说，设苹果的价格为 x ，支付金额为 L ，则相当于求 $\frac{dL}{dx}$ 。当苹果的价格稍微上涨时，支付金额会增加多少。

反向传播的原理是，从输出端开始，沿着计算图从右向左，将每个节点的输入值乘以对应的局部导数，逐步传递下去。在这个例子中，反向传播从右向左传递导数的值 ($1 \rightarrow 1.1 \rightarrow 2.2$)。从这个结果中可知，“支付金额关于苹果的价格的导数”的值是 2.2。这意味着，如果苹果的价格上涨 1 元，最终的支付金额会增加 2.2 元。可以看出，计算图的优点是，可以通过反向传播高效地计算各个变量的导数值。

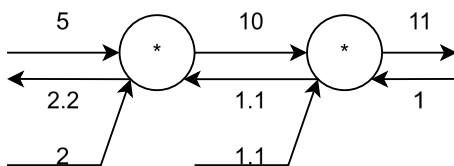


图 2.2: 反向传播示例

计算图的反向传播是基于链式法则成立的。以刚才的苹果问题为例，设苹果的价格为 x ，则计算过程可以分解为两步：

$$z = 2x \quad (\text{苹果总价})$$

$$L = 1.1z \quad (\text{含税价格})$$

由链式法则：

$$\frac{dL}{dx} = \frac{dL}{dz} \cdot \frac{dz}{dx} = 1.1 \times 2 = 2.2$$

可以看出，反向传播中每一步传递的恰好就是每个节点的局部导数 ($\frac{dL}{dz} = 1.1$, $\frac{dz}{dx} = 2$)，将它们沿计算路径依次相乘，就得到了最终的导数值。

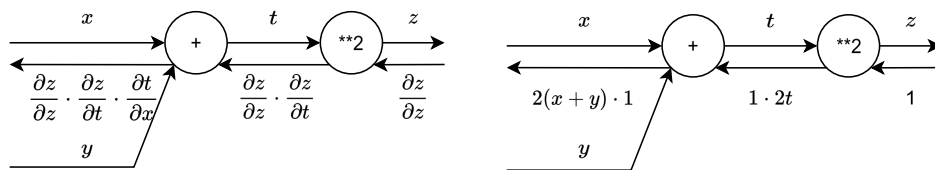


图 2.3: 反向传播示例

2.3 参数优化方法

通过反向传播算法，我们可以高效地计算损失函数关于每个参数的梯度。然而，如何利用这些梯度来更新参数，使得损失函数能够尽可能快地收敛到最小值，这就是参数优化方法要解决的问题。

2.3.1 SGD

随机梯度下降（Stochastic Gradient Descent）是最基本的优化方法。在第 1 章中，我们介绍的梯度法更新公式实际上就是 SGD，其核心思想是沿着梯度的反方向更新参数，更新规则为：

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L$$

其中， θ 是参数， η 是学习率， $\nabla_{\theta} L$ 是损失函数关于参数的梯度。

SGD 的优点是简单、计算开销小，但 SGD 也存在一些问题。在损失函数的等高线呈狭长椭圆形（即不同方向上曲率差异很大）时，梯度方向会频繁地指向山谷两侧，导致参数在山谷两侧来回震荡，即呈“之”字形移动，而沿山谷方向的进展却十分缓慢。

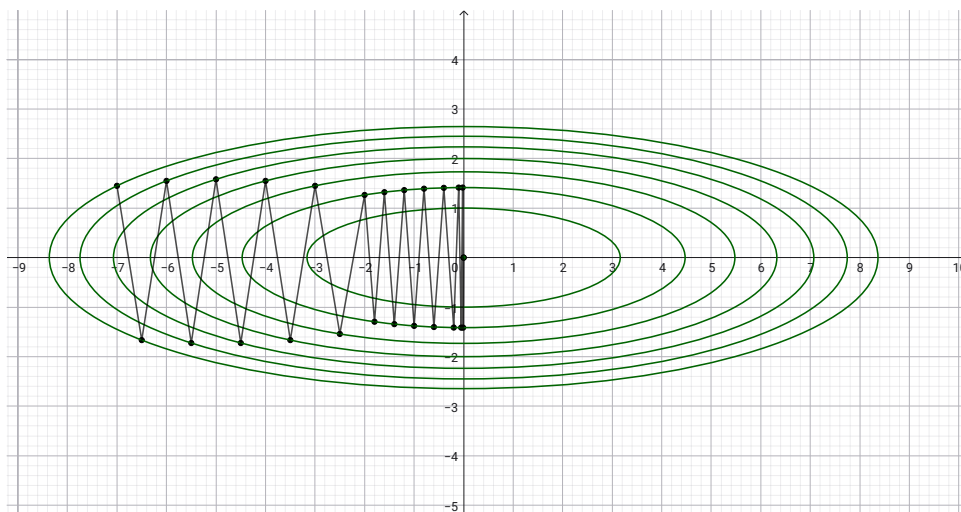


图 2.4: SGD 在 $f(x, y) = \frac{1}{10}x^2 + y^2$ 上的优化轨迹

2.3.2 Momentum

动量法（Momentum）在 SGD 的基础上引入了“速度”的概念，模拟物理学中的惯性。其更新规则为：

$$v \leftarrow \alpha v + \eta \nabla_{\theta} L$$

$$\theta \leftarrow \theta - v$$

其中， v 是速度， α 是动量衰减参数。

这个更新规则可以用物理中的小球滚动来直观理解：把损失函数想象成一个凹凸不平的地面，参数更新的过程就像一个小球在其上滚动。式中的 $\eta \nabla_{\theta} L$ 相当于小球受到的滚动的力（沿坡度方向），而 v 是小球当前的速度， αv 项表示速度的惯性——小球会保留一部分之前的运动状态。当物体不受任何力时， αv 这一项承担使物体逐渐减速的任务，对应物理上的地面摩擦或空气阻力。因此，当梯度方向持续一致时，小球越滚越快；当梯度方向频繁变化时（如狭长山谷），惯性会抑制来回震荡，使小球更稳定地朝谷底前进。

2.3.3 AdaGrad

在神经网络的学习中，学习率的值很重要。学习率过小，会导致学习花费过多时间；反过来，学习率过大，则会导致学习发散而不能正确进行。在关于学习率的优化中，有一种被称为学习率衰减（learning rate decay）的方法，即随着学习的进行，使学习率逐渐减小。实际上，一开始“多”学，然后逐渐“少”学的方法，在神经网络的学习中经常被使用。

AdaGrad（Adaptive Gradient）发展了这种想法，它为每个参数维护一个独立的学习率。对于梯度较大的参数，学习率会相应减小；对于梯度较小的参数，学习率会相应增大。其更新规则为：

$$\begin{aligned} G &\leftarrow G + (\nabla_{\theta} L)^2 \\ \theta &\leftarrow \theta - \frac{\eta}{\sqrt{G + \epsilon}} \nabla_{\theta} L \end{aligned}$$

其中， G 是历史梯度平方的累积和， ϵ 是一个极小值（如 10^{-8} ），用于防止分母为零。

2.3.4 RMSprop

AdaGrad 虽然实现了自适应学习率，但它有一个明显的缺点：随着训练的进行， G 会不断累积所有历史梯度的平方和，导致学习率单调递减，最终可能过早停止学习。为了改善这个问题，可以使用 RMSprop（Root Mean Square Propagation）方法：不再累积所有历史梯度的平方和，而是使用指数加权移动平均来只关注最近的梯度信息。其更新规则为：

$$\begin{aligned} G &\leftarrow \rho G + (1 - \rho)(\nabla_{\theta} L)^2 \\ \theta &\leftarrow \theta - \frac{\eta}{\sqrt{G + \epsilon}} \nabla_{\theta} L \end{aligned}$$

其中， ρ 是衰减率，控制历史信息的衰减速度。

2.3.5 Adam

Adam (Adaptive Moment Estimation) 结合了 Momentum 和 AdaGrad, 其更新规则为:

$$\begin{aligned} m &\leftarrow \beta_1 m + (1 - \beta_1) \nabla_{\theta} L \\ v &\leftarrow \beta_2 v + (1 - \beta_2) (\nabla_{\theta} L)^2 \\ \hat{m} &\leftarrow \frac{m}{1 - \beta_1^t} \\ \hat{v} &\leftarrow \frac{v}{1 - \beta_2^t} \\ \theta &\leftarrow \theta - \frac{\eta}{\sqrt{\hat{v} + \epsilon}} \hat{m} \end{aligned}$$

其中, m 是梯度的一阶矩估计 (相当于动量项), v 是梯度的二阶矩估计 (用于自适应学习率), β_1 和 β_2 是衰减系数, t 是当前迭代步数, $\hat{m}$ 和 $\hat{v}$ 是对 m 和 v 的偏差修正。

2.3.6 AdamW

在机器学习中, 为了防止模型过拟合, 我们通常会引入权重衰减 (Weight Decay)。该方法通过在学习的过程中对大的权重进行惩罚, 来抑制过拟合。具体来说, 为损失函数加上权重的 L_2 范数的平方, 用符号表示的话, 如果将权重记为 W , L_2 范数的权重衰减就是 $\frac{1}{2} \lambda W^2$, 然后将这个 $\frac{1}{2} \lambda W^2$ 加到损失函数上。这里, λ 是控制正则化强度的超参数, λ 设置得越大, 对大的权重施加的惩罚就越重。

AdamW 是 Adam 的一个改进版本, 主要解决了 Adam 在权重衰减实现上的问题。在原始的 Adam 中, 如果权重衰减被直接加到了梯度中 (即加上 λW), 会导致正则化效果不稳定。这是因为 Adam 的自适应学习率机制会为每个参数维护不同的有效学习率 $\frac{\eta}{\sqrt{\hat{v} + \epsilon}}$ 。当 λW 被加到梯度中时, 它会影响二阶矩估计 $\hat{v}$, 导致自适应学习率反过来“缩放”权重衰减的效果: 对于梯度较大的参数, $\hat{v}$ 较大, 有效学习率较小, 权重衰减的效果被削弱; 对于梯度较小的参数, $\hat{v}$ 较小, 权重衰减的效果被放大。这种不一致的行为使得不同参数受到的正则化强度不一致, 正则化效果变得不稳定。

AdamW 将权重衰减从梯度计算中解耦出来, 直接作用在参数更新上:

$$\begin{aligned} m &\leftarrow \beta_1 m + (1 - \beta_1) \nabla_{\theta} L \\ v &\leftarrow \beta_2 v + (1 - \beta_2) (\nabla_{\theta} L)^2 \\ \hat{m} &\leftarrow \frac{m}{1 - \beta_1^t} \\ \hat{v} &\leftarrow \frac{v}{1 - \beta_2^t} \\ \theta &\leftarrow \theta - \eta \left(\frac{\hat{m}}{\sqrt{\hat{v} + \epsilon}} + \lambda \theta \right) \end{aligned}$$

其中, λ 是权重衰减系数。实验表明, AdamW 在训练深度神经网络 (尤其是 Transformer 模型) 时, 通常比 Adam 具有更好的泛化性能。

2.4 归一化和正则化

2.4.1 权重初始值

如果我们把所有权重初始化为 0，那么会发现学习无法正确进行。为什么不能将权重设为 0 呢？严格来说，权重初始值不能全部设为一样的值，因为如果这样的话，每一层的神经元都会计算出相同的结果，更新相同的梯度，最终得到相同的权重——这意味着无论有多少个神经元，它们都只能学到相同的特征，这显然失去了多神经元的意义。

那么，应该使用什么样的分布来初始化权重呢？一个常见的做法是使用高斯分布进行随机初始化，但权重的尺度（scale）需要精心设计。核心目标是：让各层激活值的分布保持适当的广度。原因在于，如果各层传递的激活值分布差异过大或过于集中，信号在逐层传递的过程中会逐渐失真，导致学习困难。具体来说：

- **尺度过大**（标准差过大）：意味着初始权重矩阵中存在大量绝对值极大的数值。前向传播时，激活值会随着层数加深而剧烈放大。若使用 Sigmoid 等有界激活函数，激活值会迅速落入饱和区（在输入很大或很小时，输出几乎不变）导致**梯度消失**；若使用 ReLU 等无界激活函数，则会直接引发数值爆炸，导致反向传播时**梯度爆炸**。
- **尺度过小**（标准差过小）：由于 $y = \sum w_i x_i$ ，输入信号的方差会随着网络层数的加深呈指数级衰减，导致表现力坍塌。

Xavier 初始化是一种常见的初始化方式，它的目标是使各层激活值呈现相同广度的分布，由此推导出了合适的权重尺度。下面给出推导过程，设某一层的输入为 $x_1, x_2, \dots, x_n$ ，对应的权重为 $w_1, w_2, \dots, w_n$ ，则该层一个神经元的输出为：

$$y = w_1 x_1 + w_2 x_2 + \dots + w_n x_n$$

假设各 x_i 相互独立且同分布，各 w_i 也相互独立且同分布，且 x_i 与 w_i 相互独立，均值均为 0。则 y 的方差为：

$$\text{Var}(y) = \sum_{i=1}^n \text{Var}(w_i x_i) = \sum_{i=1}^n \text{Var}(w_i) \text{Var}(x_i) = n \cdot \text{Var}(w) \cdot \text{Var}(x)$$

为了使各层激活值的方差保持一致，即 $\text{Var}(y) = \text{Var}(x)$ ，需要：

$$n \cdot \text{Var}(w) \cdot \text{Var}(x) = \text{Var}(x) \implies \text{Var}(w) = \frac{1}{n}$$

因此，权重初始值从均值为 0、方差为 $\frac{1}{n}$ 的分布中采样：

$$W \sim N\left(0, \frac{1}{n}\right)$$

Xavier 初始化是以激活函数是线性函数（sigmoid 和 tanh 等函数在中央附近可被视为线性函数）为前提推导出来的，He 初始化则是针对 ReLU 激活函数提出的。由于 ReLU 会将负值置为零，

输出的方差会减小为原来的一半，因此需要将方差放大一倍。如果前一层的节点数为 n ，则权重从均值为 0、方差为 $\frac{2}{n}$ 的分布中采样：

$$W \sim N\left(0, \frac{2}{n}\right)$$

2.4.2 归一化

前文提到，权重初始化试图让各层激活值保持适当的广度，但这只是初始时刻的“一次性保障”——随着训练的进行，参数不断更新，各层激活值的分布会逐渐偏离初始状态。归一化则是在训练过程中**强制性地**将每层的输入拉回到一个良好的分布，具体而言，就是使数据分布均值为 0、方差为 1。

这种对分布的约束还带来了另一个好处。回顾前文参数优化方法中提到的问题：当损失函数的等高线呈狭长椭圆形时，梯度方向会频繁地指向山谷两侧，导致必须使用极小的学习率慢吞吞地下山。而归一化将各层输入缩放到相似的区间，使损失函数的等高线变得接近“正圆形”，梯度方向直接指向最小值，允许使用更大的学习率，从而大幅加速收敛。常用的归一化方法有 Batch Normalization(BN) 和 Layer Normalization(LN)。

Batch Normalization，顾名思义，以进行学习时的 mini-batch 为单位进行归一化。对于一个 mini-batch 中的数据 $\{x_1, x_2, \dots, x_m\}$ （同一特征在不同样本上的取值），计算过程如下：

$$\begin{aligned}\mu_B &\leftarrow \frac{1}{m} \sum_{i=1}^m x_i \\ \sigma_B^2 &\leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2 \\ \hat{x}_i &\leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \\ y_i &\leftarrow \gamma \hat{x}_i + \beta\end{aligned}$$

其中， μ_B 是 mini-batch 的均值， σ_B^2 是 mini-batch 的方差， $\hat{x}_i$ 是归一化后的值， γ 和 β 是可学习的缩放和平移参数， ϵ 是一个极小值，用于防止分母为零。

与 BN 不同，Layer Normalization 不依赖 batch 维度，而是对单个样本在该层的所有神经元输出进行标准化。对于一个样本的激活值 $\{a_1, a_2, \dots, a_d\}$ （同一层中 d 个神经元的输出），计算过程如下：

$$\begin{aligned}\mu_L &\leftarrow \frac{1}{d} \sum_{i=1}^d a_i \\ \sigma_L^2 &\leftarrow \frac{1}{d} \sum_{i=1}^d (a_i - \mu_L)^2 \\ \hat{a}_i &\leftarrow \frac{a_i - \mu_L}{\sqrt{\sigma_L^2 + \epsilon}} \\ y_i &\leftarrow \gamma \hat{a}_i + \beta\end{aligned}$$

其中, μ_L 是该层所有神经元输出的均值, σ_L^2 是方差, γ 和 β 同样是可学习的缩放和平移参数。

LN 与 BN 的关键区别在于归一化的方向不同: BN 沿 batch 维度归一化 (同一特征跨样本), LN 沿特征维度归一化 (同一层跨神经元)。因此, LN 不依赖 batch size, 适用于 batch size 较小的场景, 也是 Transformer 等模型的标准配置。

2.4.3 正则化

正则化 (Regularization) 是一类用于抑制过拟合、提升泛化能力的技术的统称。除了前文介绍的权重衰减 (L2 正则化), 常用正则化方法还有 Dropout 和数据增强 (Data Augmentation)。

Dropout 是一种在学习过程中随机删除神经元的方法, 具体来说, 在每次前向传播时, Dropout 会以概率 p 随机选择神经元并将其输出置为零, 其余神经元的输出则除以 $1 - p$ 进行缩放。在测试时, Dropout 不再随机关闭神经元, 所有神经元都参与计算。

数据增强则是从数据层面入手来抑制过拟合: 通过对训练数据施加各种变换 (如旋转、翻转、裁剪、颜色抖动等), 生成更多样化的训练样本, 从而增加数据的有效规模, 迫使模型学习更具泛化性的特征。

第三章 CNN

3.1 CNN 简介

本章的主题是卷积神经网络 (Convolutional Neural Network, CNN), 之前介绍的神经网络中, 相邻层的所有神经元之间都有连接, 这称为全连接网络 (fully-connected Network)。全连接网络主要由全连接层组成, 其本质上是线性变换加偏置 ($y = Wx + b$), 所以也叫做 Affine 层。

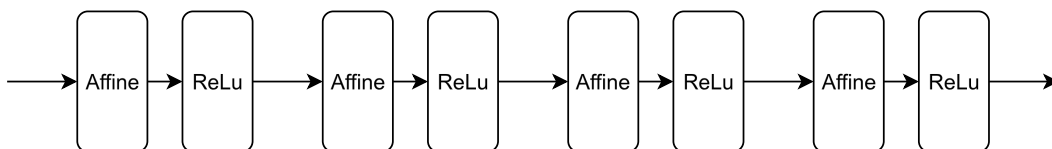


图 3.1: 全连接网络示例

相较全连接网络, CNN 中新出现了卷积层 (Convolution 层) 和池化层 (Pooling 层), CNN 的连接顺序是 “Conv - ReLU - Pooling” (Pooling 层有时会被省略)。还需要注意的是, 在 CNN 中, 靠近输出的层中使用了之前的 “Affine - ReLU” 组合。此外, 最后的输出层中使用了之前的 “Affine - Softmax” 组合, 这些都是 CNN 中比较常见的结构。

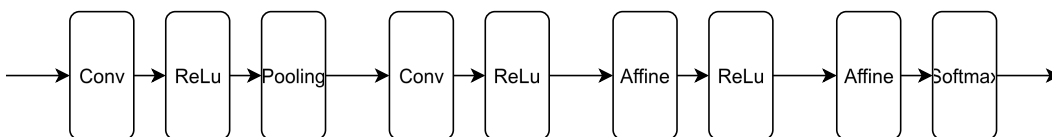


图 3.2: 卷积神经网络示例

3.2 卷积层

在介绍卷积层之前, 我们先来看看全连接层 (Affine 层) 在处理图像数据时的局限。图像通常是高、长、rgb 通道方向上的 3 维形状, 其中包含了重要的空间信息——相邻像素往往具有相似的值, 形状中也可能隐藏着值得提取的特征。然而, 向全连接层输入图像时, 需要将 3 维数据拉平为 1 维数据, 这些空间信息通通被 “忽视” 了。

为了解决这个问题, CNN 引入了卷积层。卷积层可以保持形状不变, 当输入数据是图像时, 卷积层会以 3 维数据的形式接收输入数据, 并同样以 3 维数据的形式输出至下一层。利用卷积层, 有

可能正确理解输入数据的形状的信息。卷积层的输入输出数据称为特征图 (feature map)，其中，输入数据称为输入特征图 (input feature map)，输出数据称为输出特征图 (output feature map)。

卷积层进行的处理就是卷积运算，相当于图像处理中的“滤波器运算”。具体来说，卷积运算会以一定间隔滑动滤波器（也称为卷积核）的窗口，对窗口内的元素进行乘积累加运算。

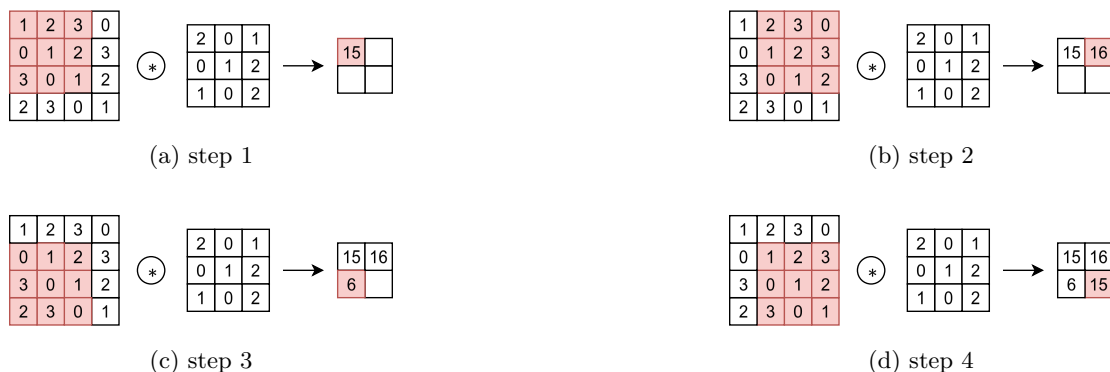


图 3.3: 卷积运算流程说明

CNN 中也存在偏置，不过偏置通常只有 1 个，这个值会被加到应用了滤波器的所有元素上。

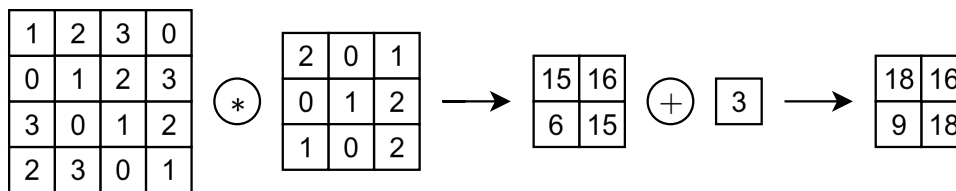


图 3.4: 偏置示例

在进行卷积层的处理之前，有时要向输入数据的周围填入固定的数据（比如 0），这称为填充 (padding)，主要是为了调整输出的大小。如图 3.5 所示，通过应用幅度为 1 的填充，(4, 4) 的输入数据变成了 (6, 6) 的形状，然后，应用形状为 (3, 3) 的滤波器，生成了形状为 (4, 4) 的输出数据。

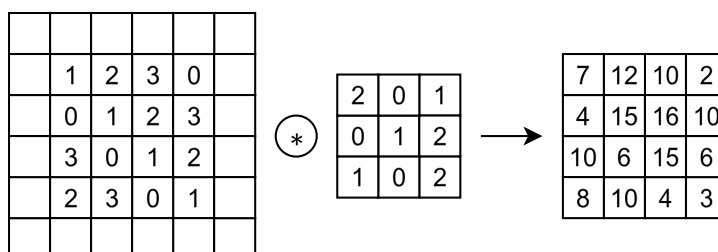


图 3.5: 填充示例

应用滤波器的位置间隔称为步幅 (stride)，图 3.6 为一个步幅为 2 的卷积运算示例。



图 3.6: 步幅示例

图像是 3 维数据，除了高、长方向之外，还需要处理通道方向，在 3 维数据的卷积运算中，输入数据和滤波器的通道数要设为相同的值。最后数据输出是 1 张特征图，如果要数据输出在通道方向上也拥有维度，那么就需要用到多个滤波器，生成多个特征图。

3.3 池化层

经过卷积层之后，特征图的空间尺寸可能仍然很大，这会导致后续层的参数量和计算量过多。池化层的作用就是缩小高、长方向上的空间尺寸，从而减少计算量，同时保留主要的特征信息。下图是按步幅 2 进行 2×2 的 Max 池化时的处理顺序。一般来说，池化的窗口大小会和步幅设定成相同的值。除了 Max 池化之外，还有 Average 池化、随机池化等，在图像识别领域，主要使用 Max 池化。

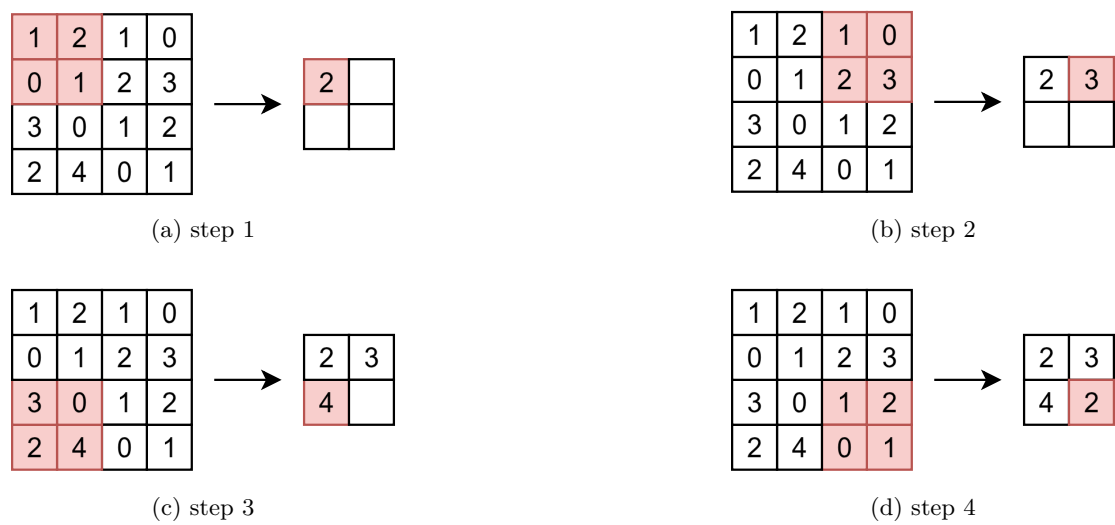


图 3.7: 池化层流程说明

池化层有几个重要特征。首先，和卷积层不同，池化层没有要学习的参数。其次，经过池化运算，输入数据和输出数据的通道数不会发生变化。此外，池化层对微小的位置变化具有鲁棒性——即使输入数据发生微小偏差，池化仍会返回相同的结果。

3.4 CNN 的发展

3.4.1 AlexNet

AlexNet 由 Alex Krizhevsky 等人于 2012 年提出，在 ILSVRC 这个大规模图像识别比赛中以显著优势夺冠，标志着深度学习在计算机视觉领域的崛起。其主要创新包括：使用 ReLU 激活函数替代传统的 Sigmoid，缓解了梯度消失问题；引入 Dropout 随机失活来抑制过拟合；以及利用 GPU 进行并行训练加速。网络结构上，AlexNet 包含 5 个卷积层和 3 个全连接层，整体较之前的网络更深。

3.4.2 VGG

VGG 由牛津大学视觉几何组 (Visual Geometry Group) 于 2014 年提出，其核心思想是通过堆叠多个小卷积核 (3×3) 来替代单个大卷积核 (如 5×5 、 7×7)。这样做有两个好处：一方面，多个 3×3 卷积核的感受野与一个大卷积核相同，但参数量更少、计算更高效；另一方面，更多的非线性层增强了网络的表达能力。VGG 网络 (如 VGG16、VGG19) 通过这种简洁统一的设计，将网络深度提升到了 16 层甚至 19 层，在 ILSVRC 中取得了优异成绩。

3.4.3 GoogLeNet

GoogLeNet 同样在 2014 年的 ILSVRC 中表现出色，但其设计思路与 VGG 不同，它的特征是，网络不仅有深度，而且有广度。实现广度的是 Inception 模块：在同一层中并行使用多种不同尺寸的卷积核 (1×1 、 3×3 、 5×5) 以及池化操作，然后将它们的输出拼接在一起。这样网络可以同时捕捉不同尺度的特征，而无需人工决定使用哪种卷积核。此外，GoogLeNet 大量使用 1×1 卷积核来进行降维，减少计算量和参数量。

3.4.4 ResNet

随着网络层数的增加，一个看似矛盾的问题出现了：更深的网络反而表现更差——训练误差和测试误差都上升了，这并非过拟合所致，而是网络难以被优化。ResNet (残差网络) 由何恺明等人于 2015 年提出，通过引入“小路” (skip connection) 巧妙地解决了这个问题。其核心思想是“残差学习”：不再直接学习目标映射 $H(x)$ ，而是学习残差 $F(x) = H(x) - x$ ，最终输出变为 $F(x) + x$ 。这种结构使得梯度可以绕过中间层直接回传，有效缓解了深层网络中的梯度消失问题。通过在 VGG 中引入这个设计，ResNet 成功训练了 152 层的超深网络，并在 ILSVRC 中大幅领先。

第二部分

自然语言处理

第四章 单词表示与分词

4.1 单词表示

自然语言处理 (Natural Language Processing, NLP), 它的目标就是让计算机理解人说的话, 进而帮助人们更好地完成某些事情。语言是由文字构成的, 含义是由单词构成的, 即单词是含义的最小单位。所以, 为了让计算机理解自然语言, 第一步就是让它理解单词含义。

4.1.1 基于同义词词典的方法

要表示单词含义, 首先可以考虑通过人工方式来定义单词含义, 一种方式就是像词典一样, 一个词一个词地说明单词含义。NLP 中广泛运用的一种词典是同义词词典 (thesaurus), 在词典中, 含义类似的单词被归类到同一个组中。比如, car 的同义词有 automobile、motorcar 等。另外, 在同义词词典中, 有时还会定义更多的关系, 比如上位下位关系、整体部分关系等。在 NLP 领域, 最著名的同义词词典是普林斯顿大学于 1985 年开始开发的同义词词典 WordNet。通过构建这样的同义词词典, 可以教会计算机单词之间的相关性。也就是说, 我们可以将单词含义 (间接地) 教给计算机。

这种人工构建的同义词词典有许多缺陷, 第一点是它难以实时更新, 随着时间的推移, 会有新词不断出现, 也会有旧词被赋予新的含义, 同义词词典很难反映出这种变化。第二点是生成同义词词典的人力成本高, 以英文为例, 据说现有的英文单词总数超过 1000 万个, 光靠人力创造完全的同义词词典不太现实。第三点是它无法表示单词的微妙差异, 含义相近的单词之间可能也会有细微的区别, 这些区别同义词词典不能很好体现。

4.1.2 基于统计的方法

为了克服同义词词典方法的这些缺陷, 研究人员转向了基于统计的方法。这种方法的目标是将单词表示成向量形式, 在 NLP 中也称为分布式表示。

这个方法基于分布式假设 (distributional hypothesis): “某个单词的含义由它周围的单词形成”, 也就是说单词含义可以由它所在的上下文表示。因此, 需要从大量文本数据 (称为语料库 (corpus), 如 Wikipedia 等) 中统计单词的上下文关系。考虑句子: “I say yes and you say no.”, 首先对文本进行分词, 记录下每个单词的上下文 (假设窗口大小为 1), I 的上下文只有 say, 其表示为:

	I	say	yes	and	you	no	.
I	0	1	0	0	0	0	0

即 you 向量表示为 (0, 1, 0, 0, 0, 0, 0), 继续处理剩余单词。最后生成一个生成共现矩阵 (co-occurrence matrix), 矩阵每行对应一个单词向量, 里面的元素表示两个单词同时出现的次数:

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

这种表示有时可能出现问题。考虑语料库频繁出现 “...the car...” 的情况, 如果只看单词的出现次数, 有可能出现 the 和 car 的相关性比 car 与 drive 相关性更强的情况。为了解决这一问题, 可以使用点互信息 (Pointwise Mutual Information, PMI) 指标, PMI 的值越高, 表明相关性越强。对于单词 x 和 y , 它们的 PMI 定义如下:

$$\text{PMI}(x, y) = \log \frac{P(x, y)}{P(x)P(y)}$$

其中, $P(x)$ 和 $P(y)$ 分别是单词 x 和 y 在语料库中出现的概率 (边缘概率), $P(x, y)$ 是 x 和 y 同时出现的概率 (联合概率)。

为了避免 $\log 0$, 实践上会使用正点互信息 (Positive PMI, PPMI):

$$\text{PPMI}(x, y) = \max(0, \text{PMI}(x, y))$$

4.1.3 基于推理的方法

基于统计的方法在处理大规模语料库会出现问题, 由于单词数量非常大, 生成的矩阵也会非常庞大, 处理起来不太方便。为了将单词转换为向量表示, 有一种简单的方式是使用 one-hot 编码。这种表示方法中, 向量的长度等于词汇表的大小, 且仅将对应单词 ID 的位置设为 1, 其余位置均为 0。不过 one-hot 编码虽然简单, 但无法体现单词之间的语义关系。

基于推理的方法通过训练预测模型来学习更好的单词表示: 定义一个预测任务 (如从上下文预测目标词), 用神经网络来学习完成这个任务, 训练完成后, 模型的参数就包含了单词的分布式表示。模型的输入通常就是单词的 one-hot 向量, 经过隐藏层变换后得到低维稠密向量, 这个向量即为单词的分布式表示, 这被称为词嵌入 (word embedding), 所以输入侧中间层有时也被称为 Embedding 层。由于 one-hot 向量中只有一个元素为 1, 其他元素都为 0, 所以它与中间层权重矩阵的乘积其实就是取矩阵的某一行, 于是在实践上 Embedding 层一般用查表的方式代替矩阵乘积。

基于推理的方式的重要成果就是 Google 于 2013 开源的词向量计算工具 Word2Vec。Word2Vec 中有两个模型: CBOW (continuous bag-of-words) 模型和 skip-gram 模型 CBOW 模型的输入是

以单词列表表示的上下文，其中每个单词用 one-hot 表示成向量，所以如果上下文考虑 N 个单词，就会有 N 个输入层。在 CBOW 中，中间层的神经元是各个输入层经全连接层变换后得到的值的平均，输出层神经元数目等于词表大小，这些神经元代表各个单词的得分，它的值越大，说明对应单词的出现概率就越高。skip-gram 模型反转了 CBOW 模型的目标：CBOW 模型从上下文预测单词，而 skip-gram 模型则从单词预测上下文。无论是 CBOW 还是 skip-gram，训练完成后，输入侧 Embedding 层的权重矩阵的每一行就对应一个单词的词嵌入。

4.2 分词

为了得到文本的单词表示，还需要先分词 (Tokenization)，即将文本切分成一个个 Token，Token 可以是单词、子词或字符等。分词的方式有很多种，比较简单的方式是按字、按词切分，复杂一点的方式是 n -gram 分词，切分时 n 个 Token 为一组来对文本进行切分。

- 文本：人工智能让世界变得更美好
- 按字：人 工 智 能 让 世 界 变 得 更 美 好
- 按词：人 工 智 能 让 世 界 变 得 更 美 好
- 按字 Bi-Gram：人/工 工/智 智/能 能/让 让/世 世/界 界/变 变/得 得/更 更/美 美/好 好/
- 按词 Bi-Gram：人工智能/让 让/世界 世界/变得 变得/更 更/美好 美好/

无论采用哪种分词方式，都需要一个词表来定义所有合法的 Token，因此，词表的构建就是分词的关键，由此产生了多种分词算法，常见的有 BPE、BBPE、WordPiece 等。

4.2.1 BPE

先介绍 BPE (byte pair encoding) 的工作原理，假设有一个语料库，我们首先从其中提取所有的单词并计算它们出现的次数。假设提取出来的单词和计数是：

(cost, 2)、(best, 2)、(menu, 1)、(men, 1)、(camel, 1)

先将所有的单词变成字符序列，用出现过的所有字符初始化词表，此时词表的大小为 11。

a, b, c, e, l, m, n, o, s, t, u

接下来，我们需要定义词表迭代的终止条件，假设我们要创建大小为 14 的词表。为了在词表中添加一个新 Token，我们需要先确定出现得最频繁的符号对，将它们合并之后添加到词表中，重复这一步骤，直到达到词表的大小要求。

延续前面的例子，从字符序列可以看出，出现得最频繁的符号对是 s 和 t，符号对 s 和 t 出现了 4 次（两次是在 cost 中，另外两次是在 best 中），因此将 st 添加到词表。

a, b, c, e, l, m, n, o, s, t, u, st

下面，重复同样的步骤，再次检查出现的最频繁的符号对，可以计算出现在出现得最频繁的符号对是 m 和 e，这个符号对出现了 3 次 (menu、men、camel)。将 m 和 e 合并，并将其添加到词表中，继续以上步骤，直到词表大小为 14，最终词表如下所示：

a, b, c, e, l, m, n, o, s, t, u, st, me, men
--

BPE 所涉及的步骤小结如下：

1. 从给定的语料集中提取单词并计算它们出现的次数。
2. 确定词表的大小。
3. 将单词拆分成一个字符序列。
4. 将字符序列中的所有非重复字符添加到词表中。
5. 选择并合并具有高频率的符号对。
6. 重复步骤 5，直到达到步骤 2 中所设定的词表大小。

4.2.2 BBPE

BBPE (byte-level byte pair encoding, BBPE) 的工作原理与 BPE 非常相似，但它不使用字符序列，而是使用字节序列。假设输入文本只有单词 best，在 BPE 中，需要将单词转换为字符序列：b e s t，而在 BBPE 中，我们不是将单词转换为字符序列，而是将其转换为字节序列：62 65 73 74，假设输入是“你好”这个中文词组，则会被转换为字节序列：e4 bd a0 e5 a5 bd。BBPE 的好处是字节表示的通用性使其能处理任何编码的文本。

4.2.3 WordPiece

WordPiece 和 BPE 主要的不同之处在于合并对的选择方式。WordPiece 不是选择频率最高的字符对，而是对每个字符对计算一个得分，使用公式：

$$\text{score} = \frac{\text{freq_of_pair}}{\text{freq_of_first_element} \times \text{freq_of_second_element}}$$

第五章 语言模型

5.1 语言模型简介

回忆一下上文讲的 CBOW 模型，假设单词序列为 $w_1, w_2, \dots, w_T$ 且窗口大小为 1，它所做的事情就是从上下文 w_{t-1} 和 w_{t+1} 预测目标词 w_t 。当给定上下文时，目标词是 w_t 的概率为 $P(w_t | w_{t-1}, w_{t+1})$ ，CBOW 模型就是对这一后验概率进行建模，如果使用交叉熵误差，其损失函数就是 $L = -\log P(w_t | w_{t-2}, w_{t-1})$ 。在上一章我们使用 CBOW 模型的权重获得单词的表示，那么它本来的目标“从上下文预测目标词”是否能用来做点事情呢？这就得提到语言模型了。

语言模型 (language model) 是一种对句子“打分”的模型——它判断一个句子是否自然、通顺。具体来说，语言模型输出概率来衡量一个单词序列的合理性。比如，对于 “you say goodbye” 这一单词序列，语言模型给出高概率；对于 “you say good die” 这一单词序列，模型则给出低概率。语言模型常用于机器翻译和语音识别等任务，比如，语音识别系统根据人的发言生成多个句子作为候选，此时，使用语言模型，可以按照单词序列发生的可能性对候选句子进行排序。

困惑度 (perplexity) 常被用作评价语言模型的预测性能的指标，困惑度表示“概率的倒数”。假设 “you say goodbye and i say hello.” 是语料库中一个句子，而且在向语言模型传入单词 you 时会输出的概率分布中，单词是 say 的概率为 0.2，那么困惑度就是 $1/0.2=5$ 。困惑度可以解释为语言模型做下一个选择的选项的数量，在刚才的例子中，模型的困惑度是 5，这意味着下一个要出现的单词的候选个数为 5 个。在输出数据为多个的情况下，困惑度计算公式为： $Perplexity = e^L$ ，其中，L 为交叉熵损失函数的值。

用数学表示语言模型：

$$\begin{aligned} P(w_1, \dots, w_m) &= P(w_m | w_1, \dots, w_{m-1}) \cdot P(w_{m-1} | w_1, \dots, w_{m-2}) \cdots P(w_2 | w_1) \cdot P(w_1) \\ &= \prod_{t=1}^m P(w_t | w_1, \dots, w_{t-1}) \end{aligned}$$

假设我们将 CBOW 模型改造成语言模型，那么首先得将原来对称的上下文改成只有目标词左侧的上下文，这样可以近似实现语言模型：

$$P(w_1, \dots, w_m) = \prod_{t=1}^m P(w_t | w_1, \dots, w_{t-1}) \approx \prod_{t=1}^m P(w_t | w_{t-2}, w_{t-1})$$

5.2 RNN

语言模型虽然能用 CBOW 来近似实现，但存在两个明显的局限。第一，它的上下文窗口大小是固定的，只能看到有限个过去的单词，更远的信息会被丢弃。第二，它对上下文中的单词做平均，因此单词的顺序信息会被忽视，比如，(you, say) 和 (say, you) 会被作为相同的内容进行处理。循环神经网络 (Recurrent Neural Network, RNN) 可以很好地解决上述问题。

之前我们看到的神经网络都是前馈型神经网络，网络的传播方向是单向的，没有环路，虽然前馈网络结构简单、易于理解，但它不能很好地处理时间序列数据。RNN 与前馈网络的关键区别在于它引入了环路，使得网络能够维持一个内部状态，从而捕捉序列中的时序依赖关系。

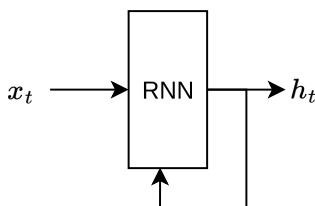


图 5.1: RNN 层示例

RNN 通过环路，使得数据可以在层内循环。时序数据 $(x_0, x_1, \dots, x_t, \dots)$ 被输入到层中。然后，以与输入对应的形式，输出 $(h_0, h_1, \dots, h_t, \dots)$ 。RNN 有两个权重，分别是转化输入 x 的权重 W_x 和转化前一个 RNN 层的输出的权重 W_h 。RNN 的计算用数学表示为： $h_t = \tanh(h_{t-1}W_h + x_tW_x + b)$ 。RNN 的输出 h_t 称为隐藏状态 (hidden state) 或隐藏状态向量 (hidden state vector)。

由于 RNN 层有环路，因此不能直接使用前馈神经网络的反向传播算法，RNN 的反向传播被称为基于时间的反向传播 (Back Propagation Through Time, BPTT)，具体做法是展开 RNN 的循环，把它看作普通的神经网络，然后运用之前的反向传播方法即可。

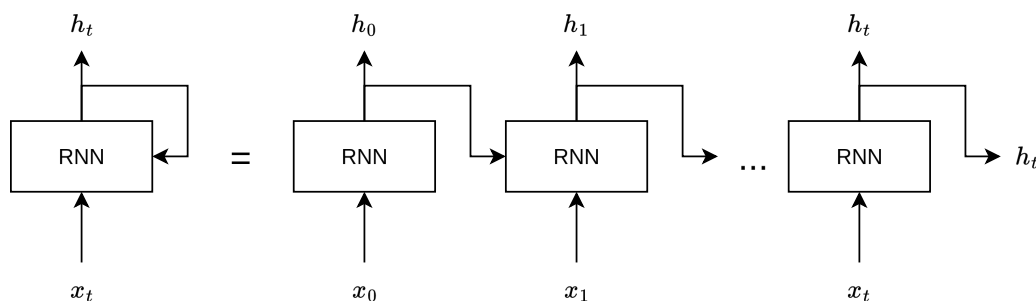


图 5.2: RNN 的展开

在处理长度为 1000 的时序数据时，如果展开 RNN 层，它将成为在水平方向上排列有 1000 个层的网络，序列太长的话，会出现计算量或者内存使用量方面的问题，此外，随着层变长，梯度逐渐

变小，最终将无法传递下去。因此，考虑将时间轴方向上过长的网络在合适的位置进行截断，从而创建多个小型网络，然后对截出来的小型网络各自执行误差反向传播法，这个方法称为 Truncated BPTT。

在 RNN 的学习中，为了执行 mini-batch 学习，需要将输入数据进行“偏移”。对长度为 1000 的时序数据，如果将批大小设为 2，那么第 1 笔样本数据从头开始按顺序输入，第 2 笔数据就从第 500 个数据开始按顺序输入。

利用 RNN 可以构建基于神经网络的语言模型，称为 RNN 语言模型 (RNN Language Model, RNNLM)。RNNLM 的结构很简单：输入层接收单词的向量表示（如词嵌入），经过 RNN 层处理后得到隐藏状态，最后通过全连接层和 softmax 输出下一个单词的概率分布。

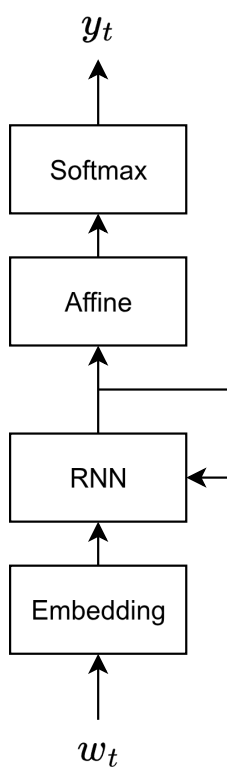


图 5.3: RNNLM

5.3 LSTM 和 GRU

前面介绍的简单的 RNN 在无法很好地学习到时序数据的长期依赖关系。RNN 中使用了函数 $\tanh(x)$ ，它的导数为 $1 - \tanh^2(x)$ ，这个值小于 1，并且随着 x 远离 0，它的值在变小，这就导致 RNN 在处理长序列时容易出现梯度消失的问题。另外，RNN 中使用了矩阵乘积，有可能导致梯度消失或梯度爆炸，这里不具体分析了。

梯度爆炸的问题比较好解决，可以通过梯度裁剪（gradients clipping）来解决，

$$g \leftarrow \begin{cases} g, & \text{if } \|g\|_2 \leq \tau \\ g \cdot \frac{\tau}{\|g\|_2}, & \text{if } \|g\|_2 > \tau \end{cases}$$

这里假设可以将神经网络用到的所有参数的梯度整合成一个向量 g ，且阈值设置为 τ 。

梯度消失的问题则更为棘手，为了解决这个问题，LSTM（Long Short-Term Memory）引入了一种名为“门”的结构，并且保存专门的权重参数用于控制门的开合程度，这些权重参数也会在学习中被更新。

在 LSTM 中存在记忆单元 c ，仅在 LSTM 层内部接收和传递数据，不向其他层输出。其中 $h_t = \tanh(c_t)$ 。

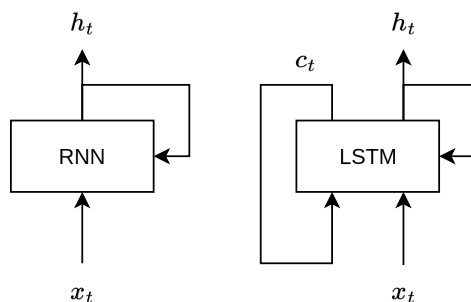


图 5.4: RNN 与 LSTM

输出门（output gate）针对 $\tanh(c_t)$ 的元素，调整它们作为下一时刻的隐藏状态的重要程度。输出门的开合程度根据输入 x_t 和上一个状态 h_{t-1} 求出（ σ 表示 sigmoid 函数），最后将这个 o_t 和 $\tanh(c_t)$ 的对应元素的乘积作为 h_t 输出。：

$$o_t = \sigma(x_t W_x^{(o)} + h_{t-1} W_h^{(o)} + b^{(o)})$$

遗忘门（forget gate）是一个忘记不必要记忆的门，遗忘门的输出 f_t 与记忆单元 c_{t-1} 相乘，生成新的记忆单元 c_t ：

$$f_t = \sigma(x_t W_x^{(f)} + h_{t-1} W_h^{(f)} + b^{(f)})$$

遗忘门用来遗忘记忆，现在我们还想向记忆单元添加一些应当记住的新信息，为此我们添加新的 $\tanh$ 节点，通过将这个 g_t 加到上一时刻的 c_{t-1} 上，从而形成新的记忆：

$$g_t = \tanh(x_t W_x^{(g)} + h_{t-1} W_h^{(g)} + b^{(g)})$$

最后添加一个输入门（input gate），将 i_t 和 g_t 的对应元素的乘积添加到记忆单元中：

$$i_t = \sigma(x_t W_x^{(i)} + h_{t-1} W_h^{(i)} + b^{(i)})$$

在 LSTM 中，记忆单元的更新仅有 + 和 $\times$ 运算，因此梯度消失和梯度爆炸问题都得到有效缓解。

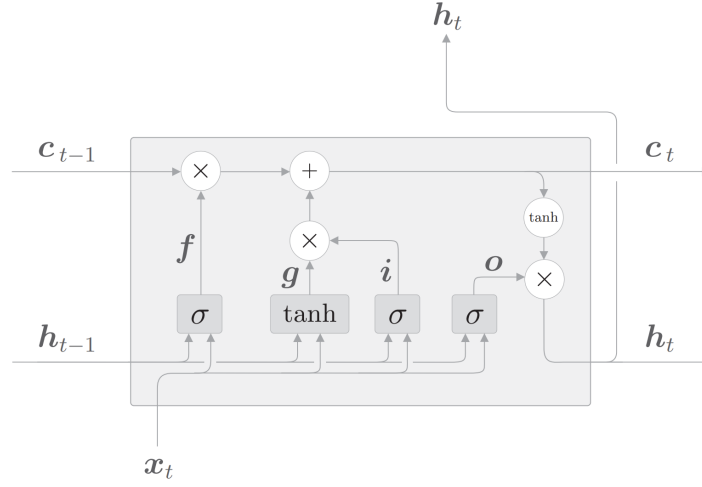


图 5.5: LSTM

GRU (Gated Recurrent Unit) 是 LSTM 的一种简化变体, 同样通过门控机制来解决梯度消失问题。与 LSTM 不同, GRU 没有记忆单元 c , 只有一个隐藏状态 h 在时间方向上传播。GRU 使用两个门 (LSTM 使用三个门): reset 门 r 和 update 门 z 。

reset 门决定在多大程度上“忽略”过去的隐藏状态:

$$r_t = \sigma(x_t W_x^{(r)} + h_{t-1} W_h^{(r)} + b^{(r)})$$

update 门扮演了 LSTM 中 forget 门和 input 门两个角色, 控制隐藏状态的更新:

$$z_t = \sigma(x_t W_x^{(z)} + h_{t-1} W_h^{(z)} + b^{(z)})$$

基于 reset 门, 计算候选隐藏状态 $\tilde{h}_t$:

$$\tilde{h}_t = \tanh(x_t W_x + (r_t \odot h_{t-1}) W_h + b)$$

如果 r_t 为 0, 则候选隐藏状态仅取决于当前输入 x_t , 过去的隐藏状态被完全忽略。

最终, GRU 通过 update 门对过去的隐藏状态和候选隐藏状态进行加权, 得到新的隐藏状态:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

其中 $(1 - z_t) \odot h_{t-1}$ 部分充当 forget 门的功能, 从过去的隐藏状态中删除应该被遗忘的信息; $z_t \odot \tilde{h}_t$ 部分充当 input 门的功能, 对新增的信息进行加权。

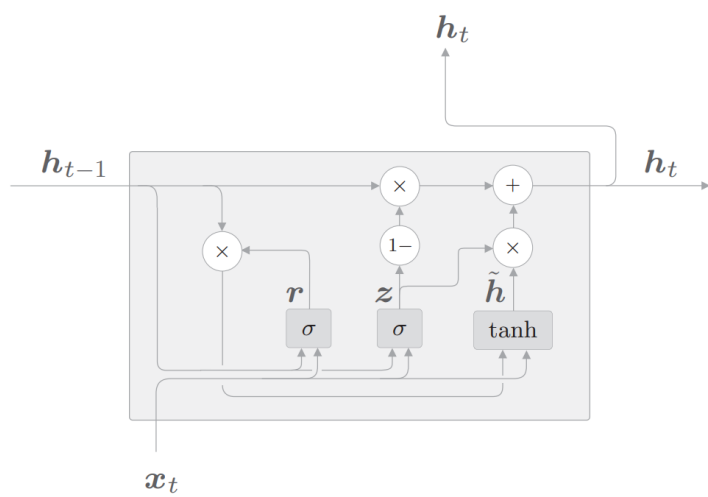


图 5.6: GRU

第六章 Transformer

6.1 Seq2Seq 模型

语言模型根据已经出现的单词输出下一个出现的单词的概率分布，通过对这个概率分布进行采样得到下一个词，再将已生成的词序列不断送回到模型中，重复该过程，直到生成结束符号 $\langle \text{eos} \rangle$ 为止，就可以生成完整的文本。

在上述过程中，如何选择下一个词取决于具体的采样策略。最简单的策略是贪心搜索 (Greedy Search)，即每次选择概率最高的词作为下一个输出，但这种方法容易生成重复或单调的文本。为了增加生成的多样性，可以采用以下策略：

- **温度采样 (Temperature Sampling)**：引入温度参数 T ，将 softmax 的输入除以 T 。当 $T < 1$ 时，概率分布更尖锐，模型更倾向于选择高概率的词；当 $T > 1$ 时，分布更平坦，生成的随机性更大。
- **Top-k 采样**：只保留概率最高的 k 个词，将其余词的概率置为零后再进行采样。
- **Top-p 采样 (Nucleus Sampling)**：动态选择一个最小的词集，使其累积概率之和达到阈值 p ，然后在该词集中进行采样。与 Top-k 不同，Top-p 的词集大小是动态变化的。

Seq2Seq 模型则是另一类文本生成模型，用于将一种序列转换为另一种序列，广泛应用于机器翻译、语音识别等任务。它通常由两个模块组成：Encoder 和 Decoder，因此也被称为 Encoder-Decoder 模型。编码器常见结构是 Embedding 层加 RNN 层，用于将输入序列编码为一个向量表示；解码器则根据这个向量表示逐步生成输出序列，其生成过程可以参考前面的采样策略。

编码器的输出通常是一个固定长度的向量，但输入序列的长度是可变的，将所有信息压缩到一个固定向量中可能导致部分信息丢失。一个自然的想法是保留编码器在每个时间步的隐藏状态，将它们拼接成一个矩阵，让解码器能够利用全部时间步的信息。然而，解码器的输入是单个向量，因此需要先对矩阵的行向量进行加权平均，将其“压缩”为单个向量。而如何确定加权平均中的权重，就是注意力机制 (Attention Mechanism) 所要解决的问题。

6.2 Self-Attention

在基于 RNN 的 Seq2Seq 模型中，注意力机制使得解码器在生成每个输出单词时，能够动态地关注输入序列中的不同部分。例如，在将 “I love you” 翻译为 “我爱你” 的过程中，生成 “我” 时模

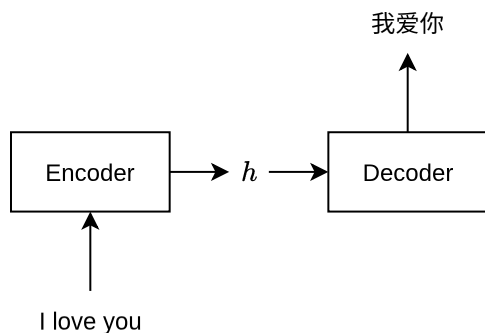


图 6.1: Seq2Seq 模型示例

型更倾向于关注 “I”，而生成 “爱” 时则更多关注 “love”。既然输出序列可以使用注意力机制，那么输入序列是否也可以使用注意力机制呢？

答案是肯定的，这正是 Transformer 模型中自注意力机制（Self-Attention）的作用。需要说明的是，Transformer 作为 Seq2Seq 模型的一种实现方式，也包含类似于基于 RNN 的 Seq2Seq 模型中的注意力机制，这部分在 Transformer 中被称为交叉注意力（Cross-Attention）。让我们通过一个例子来快速理解自注意力机制，例句：

A dog ate the food because it was hungry

显然，例句中的代词 it 指代的是 dog 而不是 food，自注意力机制的作用就是关注句子间单词的关系。

将句子输入模型的编码器，得到一个输出矩阵 $\mathbf{X}$ ，为了计算自注意力，我们需要再创建三个新的矩阵：查询（query）矩阵 $\mathbf{Q}$ 、键（key）矩阵 $\mathbf{K}$ ，以及值（value）矩阵 $\mathbf{V}$ 。为了创建它们，我们需要先创建另外三个权重（weight）矩阵， $\mathbf{W}^Q$ 、 $\mathbf{W}^K$ 、 $\mathbf{W}^V$ 。用矩阵 $\mathbf{X}$ 分别乘以矩阵 $\mathbf{W}^Q$ 、 $\mathbf{W}^K$ 、 $\mathbf{W}^V$ ，依次创建出 $\mathbf{Q}$ 、 $\mathbf{K}$ 和 $\mathbf{V}$ 。值得注意的是，权重矩阵 $\mathbf{W}^Q$ 、 $\mathbf{W}^K$ 和 $\mathbf{W}^V$ 的初始值完全是随机的，但最优值则需要通过训练获得。我们取得的权值越优，通过计算所得的查询矩阵、键矩阵和值矩阵也会越精确。

假设输入句子为 “I am good”，自注意力机制首先要计算查询矩阵 $\mathbf{Q}$ 与键矩阵 $\mathbf{K}^T$ 的点积。

$$\mathbf{Q} \cdot \mathbf{K}^T = \begin{bmatrix} \mathbf{q}_1 \cdot \mathbf{k}_1 & \mathbf{q}_1 \cdot \mathbf{k}_2 & \mathbf{q}_1 \cdot \mathbf{k}_3 \\ \mathbf{q}_2 \cdot \mathbf{k}_1 & \mathbf{q}_2 \cdot \mathbf{k}_2 & \mathbf{q}_2 \cdot \mathbf{k}_3 \\ \mathbf{q}_3 \cdot \mathbf{k}_1 & \mathbf{q}_3 \cdot \mathbf{k}_2 & \mathbf{q}_3 \cdot \mathbf{k}_3 \end{bmatrix}$$

首先，来看 $\mathbf{Q} \cdot \mathbf{K}^T$ 矩阵的第一行，可以看到，这一行计算的是查询向量 $\mathbf{q}_1$ (I) 与所有的键向量 $\mathbf{k}_1$ (I)、 $\mathbf{k}_2$ (am) 和 $\mathbf{k}_3$ (good) 的点积。通过计算两个向量的点积可以知道它们之间的相似度。因此，通过计算查询向量 ($\mathbf{q}_1$) 和键向量 ($\mathbf{k}_1$ 、 $\mathbf{k}_2$ 、 $\mathbf{k}_3$) 的点积，可以了解单词 I 与句子中的所有单词的相似度。我们了解到，I 这个词与自己的关系比与 am 和 good 这两个词的关系更紧密，因为点积值 110 大于 90 和 80。

自注意力机制的第 2 步是将 $\mathbf{Q} \cdot \mathbf{K}^T$ 矩阵除以键向量维度的平方根 $\sqrt{d_k}$ 。这是因为当 d_k 较大

$$\mathbf{Q} \cdot \mathbf{K}^T = \begin{bmatrix} 110 & 90 & 80 \\ 70 & 99 & 70 \\ 90 & 70 & 100 \end{bmatrix}$$

表 6.1: 点积示例, 本节的结果均为假设值, 后续不再说明

时, 点积结果的方差也会较大, 使得 softmax 的输入值差异悬殊, 输出接近 one-hot 分布, 导致梯度消失。除以 $\sqrt{d_k}$ 可以将点积的方差重新缩放回去, 从而获得更稳定的梯度。

目前所得的相似度分数尚未被归一化, 所以第 3 步是使用 softmax 函数对其进行归一化处理。应用 softmax 函数将使数值分布在 0 到 1 的范围内, 且每一行的所有数之和等于 1。这个矩阵被称为分数 (score) 矩阵, 通过它, 我们可以了解句子中的每个词与所有词的相关程度。比如: I 这个词与它本身的相关程度是 90%, 与 am 这个词的相关程度是 7%, 与 good 这个词的相关程度是 3%。

$$\text{softmax}\left(\frac{\mathbf{Q} \cdot \mathbf{K}^T}{\sqrt{d_k}}\right) = \begin{bmatrix} & \text{I} & \text{am} & \text{good} \\ \text{I} & 0.90 & 0.07 & 0.03 \\ \text{am} & 0.025 & 0.95 & 0.025 \\ \text{good} & 0.21 & 0.03 & 0.76 \end{bmatrix}$$

自注意力机制的最后一步是计算注意力矩阵 $\mathbf{Z}$, 注意力矩阵包含句子中每个单词的注意力向量。它通过将分数矩阵乘以值矩阵 $\mathbf{V}$ 得出。

$$\begin{aligned} \mathbf{Z} &= \text{softmax}\left(\frac{\mathbf{Q} \cdot \mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V} \\ &= \begin{bmatrix} 0.90 & 0.07 & 0.03 \\ 0.025 & 0.95 & 0.025 \\ 0.21 & 0.03 & 0.76 \end{bmatrix} \begin{bmatrix} 67.85 & 91.2 & \dots & 0.13 \\ 13.13 & 63.1 & \dots & 4.44 \\ 12.12 & 96.1 & \dots & 43.4 \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{z}_1 \\ \mathbf{z}_2 \\ \mathbf{z}_3 \end{bmatrix} \end{aligned}$$

可以看出, 单词 I 的自注意力向量 $\mathbf{z}_1$ 是分数加权的值向量之和。所以, $\mathbf{z}_1$ 的值将包含 90% 的值向量 $\mathbf{v}_1$ (I)、7% 的值向量 $\mathbf{v}_2$ (am), 以及 3% 的值向量 $\mathbf{v}_3$ (good)。

$$\mathbf{z}_1 = 0.90 \begin{bmatrix} 67.85 & 91.2 & \dots \end{bmatrix} + 0.07 \begin{bmatrix} 13.13 & 63.1 & \dots \end{bmatrix} + 0.03 \begin{bmatrix} 12.12 & 96.1 & \dots \end{bmatrix}$$

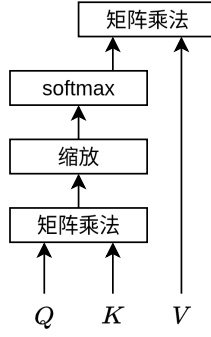


图 6.2: 自注意力计算流程

6.3 位置编码

在 RNN 中，句子是逐词送入网络的，以 I am good 为例，首先把 I 作为输入，接下来是 am，最后是 good。然而，Transformer 并不遵循这个模式，它将句子中的所有词并行地输入到神经网络中。这种方式下，词序需要通过其他方式表示，Transformer 所采用的方式是位置编码。

在 Transformer 中，位置编码以矩阵的形式表示，位置编码矩阵 $\mathbf{P}$ 的维度与输入矩阵 $\mathbf{X}$ 的维度相同。在将输入矩阵直接传给 Transformer 之前，我们需将位置编码矩阵 $\mathbf{P}$ 加到输入矩阵 $\mathbf{X}$ 中，再将其作为输入送入网络。Transformer 中使用了正弦函数来计算位置编码：

$$P(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right) \quad P(\text{pos}, 2i+1) = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

在上面的等式中，pos 表示该词在句子中的位置， i 表示在输入矩阵中的位置。

$$\mathbf{P} = \begin{bmatrix} \sin\left(\frac{\text{pos}}{10000^0}\right) & \cos\left(\frac{\text{pos}}{10000^0}\right) & \sin\left(\frac{\text{pos}}{10000^{2/4}}\right) & \cos\left(\frac{\text{pos}}{10000^{2/4}}\right) \\ \sin\left(\frac{\text{pos}}{10000^1}\right) & \cos\left(\frac{\text{pos}}{10000^1}\right) & \sin\left(\frac{\text{pos}}{10000^{2/4}}\right) & \cos\left(\frac{\text{pos}}{10000^{2/4}}\right) \\ \sin\left(\frac{\text{pos}}{10000^2}\right) & \cos\left(\frac{\text{pos}}{10000^2}\right) & \sin\left(\frac{\text{pos}}{10000^{2/4}}\right) & \cos\left(\frac{\text{pos}}{10000^{2/4}}\right) \end{bmatrix}$$

上述方法属于绝对位置编码，它将位置信息作为一个固定的向量加到输入上。另一种广泛使用的位置编码方法是旋转位置编码（Rotary Position Embedding, RoPE）。RoPE 通过对查询向量和键向量施加一个与位置相关的旋转，使得两个向量的点积自然地编码了它们之间的相对位置信息。具体来说，RoPE 将向量的相邻两个维度视为一个二维平面上的点，然后根据位置 m 旋转一个角度 $m\theta$ ：

$$\begin{bmatrix} q'_{2i} \\ q'_{2i+1} \end{bmatrix} = \begin{bmatrix} \cos m\theta_i & -\sin m\theta_i \\ \sin m\theta_i & \cos m\theta_i \end{bmatrix} \begin{bmatrix} q_{2i} \\ q_{2i+1} \end{bmatrix}$$

其中 $\theta_i = 10000^{-2i/d}$ 。经过旋转后， $\mathbf{q}_m$ 与 $\mathbf{k}_n$ 的点积只依赖于相对位置 $m-n$ ，从而使得注意力机制天然具备捕捉相对位置关系的能力。RoPE 被广泛应用于 LLaMA、Qwen 等大语言模型中。

然而，RoPE 的位置编码存在一个固有限制：它只能处理训练时见过的上下文长度。从公式可以看出，第 i 组维度的旋转角度为 $m\theta_i$ ，不同频率的维度旋转速度差异极大。高频维度（ i 较大）旋转快，低频维度（ i 较小）旋转慢，在训练时的上下文长度内，低频维度可能只旋转了极小的角度，

模型几乎没有见过更大的位置值。当推理时遇到超出训练范围的更长序列时，低频维度会进入从未探索过的区域，导致模型性能显著下降。

YaRN (Yet another RoPE extensioN) 是一种用于扩展 RoPE 上下文长度的方法。设缩放因子 $s = L'/L$ ，其中 L 为训练时的上下文长度， L' 为目标长度。定义第 i 个维度的波长为 $\lambda_i = 2\pi/\theta_i$ ，并设定两个波长阈值 λ_{low} 和 λ_{high} 。YaRN 为每个维度 i 定义一个缩放因子 s_i ，将旋转角度从 $m\theta_i$ 修改为 $m\theta_i/s_i$ ：

$$s_i = \begin{cases} 1, & \lambda_i \leq \lambda_{\text{low}} \\ s, & \lambda_i \geq \lambda_{\text{high}} \\ 1 + (s - 1) \cdot \frac{\lambda_i - \lambda_{\text{low}}}{\lambda_{\text{high}} - \lambda_{\text{low}}}, & \lambda_{\text{low}} < \lambda_i < \lambda_{\text{high}} \end{cases}$$

6.4 编码器与解码器

Transformer 的编码器由一组 N 个编码器串联而成，每一个编码器的构造都是相同的，由自注意力层和前馈网络层组成，每层的输出都经过残差连接和归一化层 (Add & Norm)。前馈网络由多个全连接层组成，一般采用 ReLU 激活函数。

相较编码器，解码器的注意力层不太一样，它需要关注编码器的输出，因此解码器的注意力层分为两部分：带掩码的注意力层和交叉注意力层。交叉注意力层都有两个输入：一个来自带掩码的注意力层，另一个是编码器的输出。让我们用 $\mathbf{R}$ 来表示编码器的输出值，用 $\mathbf{M}$ 来表示由带掩码的注意力层输出的注意力矩阵，注意力的计算使用矩阵 $\mathbf{M}$ 创建查询矩阵 $\mathbf{Q}$ ，使用 $\mathbf{R}$ 创建键矩阵和值矩阵。在解码器中的带掩码的注意层中，我们对注意力计算中的矩阵进行掩码处理，确保解码器在生成每个单词时只能关注之前的单词。

$$\frac{\mathbf{Q}_i \cdot \mathbf{K}_i^T}{\sqrt{d_k}} = \begin{bmatrix} & \text{<sos>} & \text{我} & \text{很} & \text{好} \\ \text{<sos>} & 9.125 & -\infty & -\infty & -\infty \\ \text{我} & 5.0 & 12.37 & -\infty & -\infty \\ \text{很} & 7.25 & 5.0 & 10.37 & -\infty \\ \text{好} & 1.5 & 1.37 & 1.87 & 10.0 \end{bmatrix}$$

获得解码器的输出之后，我们将其送入线性层和 softmax 层，线性层将生成一个 logit 向量，其大小等于原句中的词汇量，然后，将 softmax 函数应用于 logit 向量，从而得到概率分布，最后对其进行采样得到最终的文本输出。

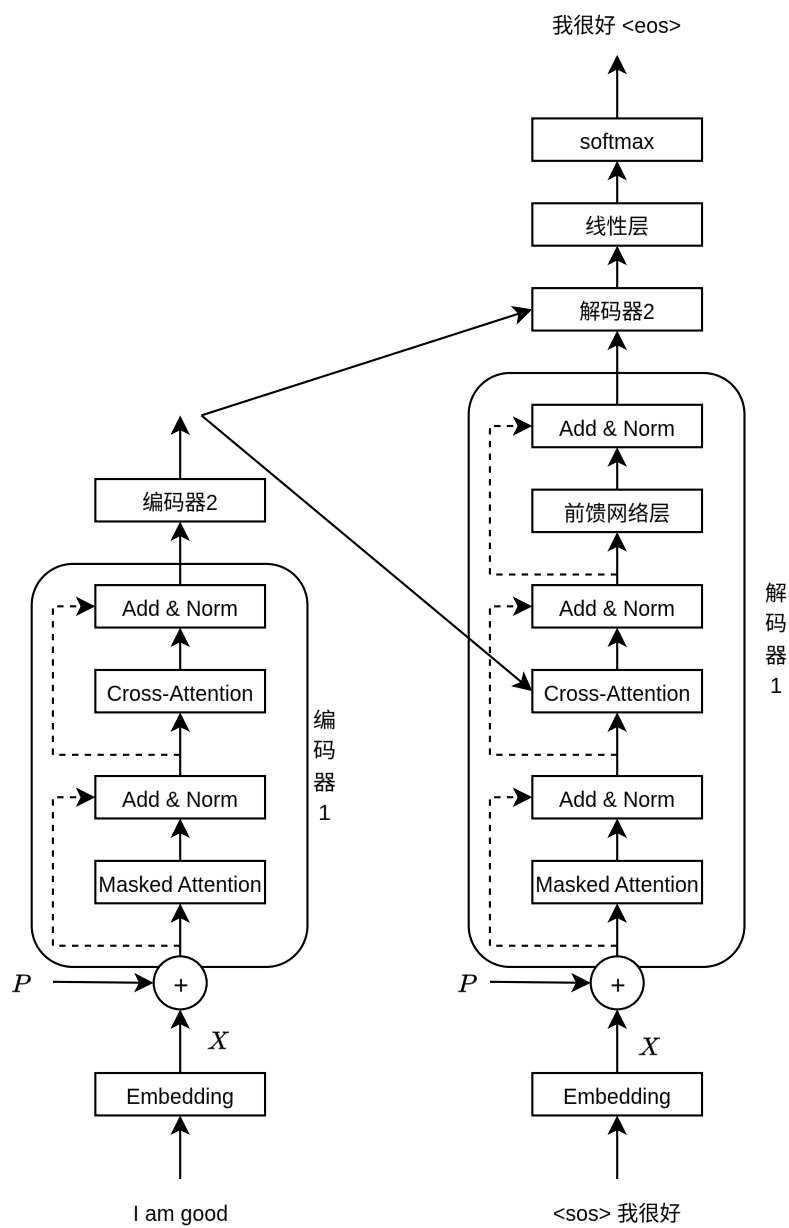


图 6.3: Transformer 总结构

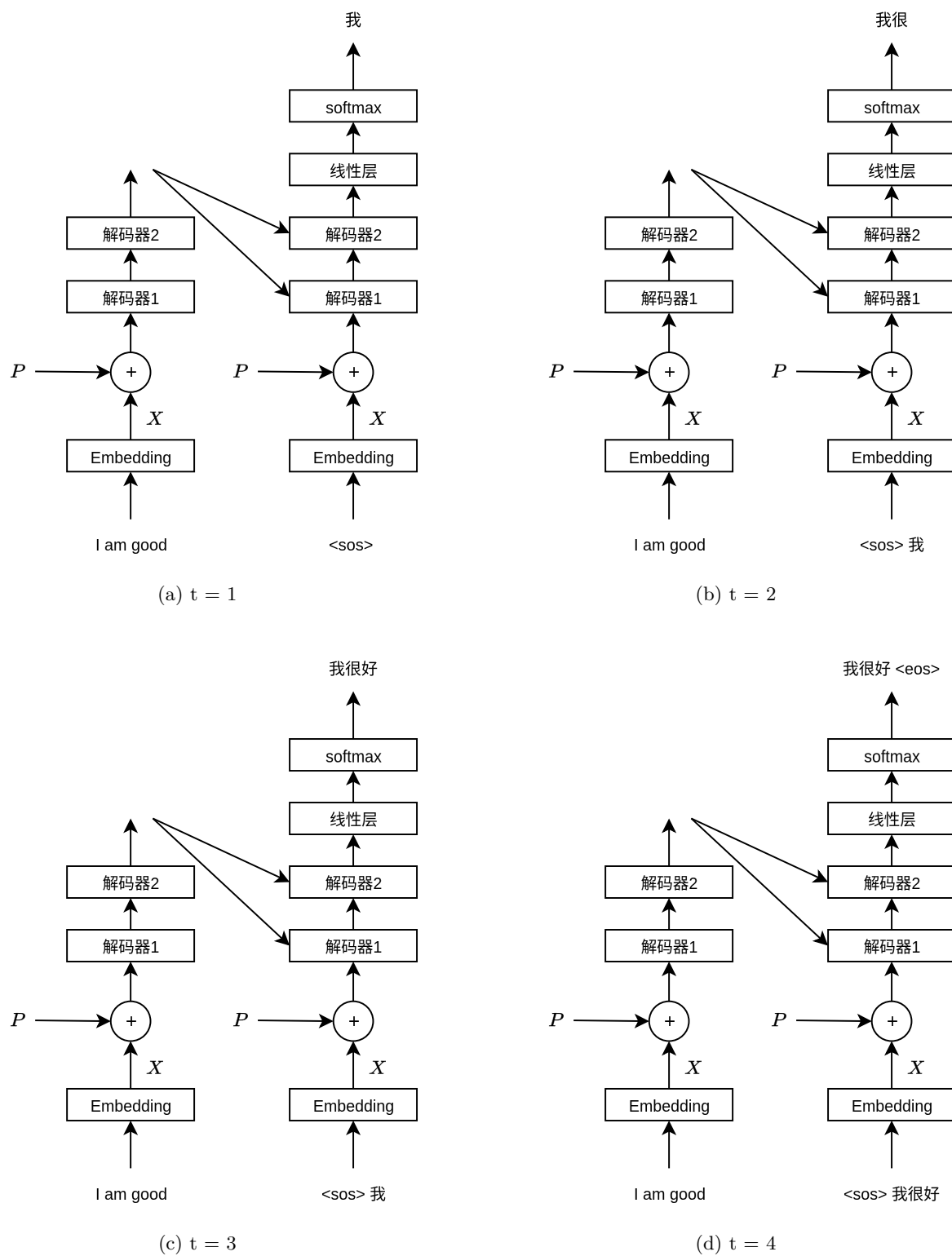


图 6.4: 解码器流程说明

第七章 注意力机制优化

在上一章中，我们学习了 Transformer 的核心机制——自注意力（Self-Attention）。标准的注意力机制在训练和推理中展现了强大的建模能力，但随着模型规模的增长和序列长度的增加，它也面临着一些实际的效率和资源问题。本章将介绍几种重要的注意力机制优化方法，它们分别从不同的角度解决这些问题。

7.1 GQA

在第 6 章中，我们学习了注意力机制。在标准的多头注意力（Multi-Head Attention, MHA）中，模型将输入送入 h 个注意力头，每个头都有独立的查询矩阵 $\mathbf{W}^Q$ 、键矩阵 $\mathbf{W}^K$ 和值矩阵 $\mathbf{W}^V$ 。在推理阶段，每个头的键和值都需要被缓存下来，以便在生成下一个 token 时复用，这就是所谓的 KV Cache。假设有 h 个注意力头，每个头的维度为 $d_h = d/h$ ，那么单个 token 对应的 KV Cache 大小为 $h \times 2 \times d_h = 2d$ 。KV Cache 的大小与序列长度 n 成正比，当序列不断变长时，KV Cache 会占用大量显存。

为了减少 KV Cache 的开销，一种直观的想法是让所有注意力头共享同一组键和值，这就是多查询注意力（Multi-Query Attention, MQA）。在 MQA 中，所有查询头共享一组键值，这样，序列长度为 n 时，KV Cache 的大小从 $2nd$ 减少到 $2nd_h$ ，减少了 h 倍。然而，由于所有头只能依赖同一组键值来计算注意力，MQA 的表达能力下降明显，模型的多样性受到了限制。

分组查询注意力（Grouped Query Attention, GQA）是 MQA 和 MHA 之间的折中方案。在 GQA 中，将 h 个查询头分成 g 组（ $1 \leq g \leq h$ ），每组共享一组键和值。当 $g = 1$ 时，GQA 退化为 MQA；当 $g = h$ 时，GQA 退化为 MHA。具体来说，假设查询头数为 h ，键值头数为 g ，每个查询头 i 属于第 $\lfloor i \cdot g/h \rfloor$ 组，使用该组的键和值来计算注意力。GQA 被广泛应用于 LLaMA3、Qwen3 等大语言模型中，它在保持模型质量的同时，显著降低了推理时的显存占用。

7.2 FlashAttention

在前面的注意力计算中，我们首先计算查询和键的点积得到注意力分数矩阵 $\mathbf{S} = \mathbf{QK}^T / \sqrt{d_k}$ ，然后对 $\mathbf{S}$ 应用 softmax 得到注意力权重矩阵 $\mathbf{P}$ ，最后乘以值矩阵 $\mathbf{V}$ 得到输出 $\mathbf{O} = \mathbf{PV}$ 。这个过程需要将 $\mathbf{S}$ 和 $\mathbf{P}$ 这两个 $n \times n$ 的矩阵存储在 GPU 的高带宽内存（HBM）中。由于 HBM 带宽有限（如 1.5 TB/s），随着序列变长，HBM 的读写开销将成为主要开销，成为性能瓶颈。

与 HBM 相比, GPU 上的 SRAM 容量虽小 (如 20MB), 但带宽极高 (如 19 TB/s)。FlashAttention 的核心思想就是通过分块计算 (tiling), 尽可能在 SRAM 中完成计算, 从而避免将 $O(n^2)$ 的中间矩阵写入 HBM。具体来说, 将 $\mathbf{Q}$ 、 $\mathbf{K}$ 、 $\mathbf{V}$ 分成若干块 (block), 每次只加载一小块到 SRAM 中计算, 计算完成后直接将结果写回 HBM, 而不需要在 HBM 中存储完整的 $\mathbf{S}$ 和 $\mathbf{P}$ 。

这里有一个关键问题: 标准 softmax 需要计算整行的指数和, 而分块计算时只能看到一部分。FlashAttention 通过 online softmax 算法解决了这个问题。具体来说, 对于注意力分数 $s_1, s_2, \dots, s_n$, 维护两个变量: 当前最大值 $m_i = \max_{j \leq i} s_j$ 、当前指数和 $d_i = \sum_{j \leq i} e^{s_j - m_i}$ 。当新增一块分数后, 新的最大值 m' 可能大于旧的最大值 m , 此时旧的指数和需要乘以 $e^{m - m'}$ 进行重新缩放:

$$d' = d \cdot e^{m - m'} + \sum_{j \in \text{新块}} e^{s_j - m'}$$

最终每个分数的注意力权重为:

$$p_i = \frac{e^{s_i - m_n}}{d_n}$$

这与一次性计算的标准 softmax 在数学上完全等价。

此外, FlashAttention 在反向传播时采用了重计算 (recomputation) 策略。在训练的反向传播过程中, 通常需要用到正向传播时产生的注意力矩阵, 但如果将其完整保存, 显存开销依然很大。FlashAttention 的做法是不保存注意力矩阵, 而是在反向传播需要用到它时, 利用 SRAM 中的分块数据和正向传播时记录的少量归一化统计量, 重新计算对应的注意力矩阵。虽然这增加了一定的计算量, 但由于省去了大量的 HBM 读写时间, 整体速度反而更快。FlashAttention 已成为大模型训练和推理的标准组件, 被集成到 PyTorch、DeepSpeed 等主流框架中。

7.3 PagedAttention

在大语言模型的推理服务中, 通常需要同时处理多个请求, 每个请求的序列长度不同且动态增长。为了支持自回归生成, 每个请求都需要维护自己的 KV Cache。传统的做法是为每个请求预分配一块连续的显存, 大小按照最大可能的序列长度来设定。然而, 由于实际序列长度远小于最大值, 且难以预测, 这种方式会导致大量的显存浪费和碎片化, 据统计, 内部碎片和外部碎片可浪费 60%–80% 的显存。

PagedAttention 借鉴了操作系统中虚拟内存的分页思想, 将 KV Cache 划分为固定大小的块 (block), 每个块包含固定数量 token 的 KV Cache (如 16 个 token)。这些块不需要在显存中连续存放, 而是通过一个块表 (block table) 来记录逻辑块到物理块的映射关系。

当生成新的 token 时, 系统按需分配新的块, 而不是一次性分配大块连续显存。这样, 显存的利用率可以接近 100%, 几乎没有浪费。此外, 由于不同请求的 KV Cache 可以共享相同的块 (例如在 beam search 或并行采样中, 多个候选序列共享前缀), PagedAttention 还可以通过块拷贝 (copy-on-write) 进一步减少显存占用, 显著提升了推理服务的吞吐量。

7.4 线性注意力

因为需要计算 $n \times n$ 的注意力分数矩阵 $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top / \sqrt{d_k}$ ，标准注意力的计算复杂度是 $O(n^2d)$ ，当序列长度达到数万甚至数十万时，计算开销就变得难以承受了。一个自然的想法是：能否通过更高效的矩阵乘法来避免这个瓶颈？

如果暂时去掉 softmax 的归一化约束，考虑简化的注意力形式 $(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$ ，利用矩阵乘法的结合律，可以改变计算顺序：

$$(\mathbf{Q}\mathbf{K}^\top)\mathbf{V} = \mathbf{Q}(\mathbf{K}^\top\mathbf{V})$$

其中 $\mathbf{K}^\top\mathbf{V} \in \mathbb{R}^{d \times d}$ ，计算复杂度 $O(nd^2)$ ； $\mathbf{Q}(\mathbf{K}^\top\mathbf{V}) \in \mathbb{R}^{n \times d}$ ，计算复杂度同样是 $O(nd^2)$ 。由于 $d \ll n$ ， $O(nd^2)$ 远优于 $O(n^2d)$ 。这就是线性注意力的核心思想——通过改变计算顺序，将复杂度从关于 n 的二次方降低到线性。

然而，softmax 引入了行归一化和指数运算，这使得计算无法简单地分解为上述结合律的形式。线性注意力的解决方案是引入核函数 κ 来替代 softmax 中的指数点积，通过设计可分解的核函数 $\kappa(\mathbf{q}_i, \mathbf{k}_j) = \phi(\mathbf{q}_i)^\top \phi(\mathbf{k}_j)$ ，将注意力输出改写为：

$$\mathbf{o}_i = \frac{\sum_{j=1}^n \kappa(\mathbf{q}_i, \mathbf{k}_j) \mathbf{v}_j}{\sum_{j=1}^n \kappa(\mathbf{q}_i, \mathbf{k}_j)} = \frac{\phi(\mathbf{q}_i)^\top \sum_{j=1}^n \phi(\mathbf{k}_j) \mathbf{v}_j}{\phi(\mathbf{q}_i)^\top \sum_{j=1}^n \phi(\mathbf{k}_j)}$$

定义聚合量 $\mathbf{S}_t = \sum_{j=1}^t \phi(\mathbf{k}_j) \mathbf{v}_j^\top \in \mathbb{R}^{d \times d}$ ， $\mathbf{z}_t = \sum_{j=1}^t \phi(\mathbf{k}_j) \in \mathbb{R}^d$ 。从形式上看，线性注意力可以看作是一种 RNN：在序列的每一步 t ，维护一个 $d \times d$ 的状态矩阵 $\mathbf{S}_t = \mathbf{S}_{t-1} + \phi(\mathbf{k}_t) \mathbf{v}_t^\top$ ，输出 $\mathbf{o}_t = \phi(\mathbf{q}_t) \mathbf{S}_t / (\phi(\mathbf{q}_t)^\top \mathbf{z}_t)$ 。这与第 5 章介绍的 RNN 结构非常相似，只是状态更新方式不同。

线性注意力的主要挑战在于如何选择合适的核函数 ϕ 来近似 softmax，使得近似误差尽可能小。常见的方法包括使用泰勒展开、随机特征近似（如 Performer）、或者通过训练学习核函数。尽管线性注意力在长序列处理上有理论优势，但在实际应用中，由于近似误差和硬件优化等因素，它通常是作为标准注意力的补充出现。

第三部分

大语言模型应用

从 Transformer 到智能体

在前面的章节中，我们从感知机一路走到了 Transformer，理解了大型语言模型的技术基础。从本章开始，我们将进入一个全新的领域——如何“用好”大模型，让它真正为我们做事。在进入具体内容之前，我们先梳理一下这项技术演进的时间脉络。

2017：Transformer 诞生

Google 发表论文《Attention Is All You Need》，提出 Transformer 架构。这个架构抛弃了 RNN 的序列处理方式，完全基于注意力机制实现并行计算，为后来所有大语言模型奠定了基石。

2020：GPT-3 与少样本学习

OpenAI 发布 GPT-3，拥有 1750 亿参数。GPT-3 最令人惊讶的能力是——它不需要微调，仅凭提示中的几个示例（Few-shot）就能完成各种任务。这让人们意识到，大模型本身就是一个通用的任务求解器，关键在于你怎么“提问”。

2022：ChatGPT 与提示词工程

2022 年底，ChatGPT 横空出世，大语言模型从实验室走向大众。与此同时，提示词工程（Prompt Engineering）成为一门显学——人们发现，同一个问题换一种问法，模型的回答质量天差地别。Zero-shot、Few-shot、Chain-of-Thought 等技术相继被提出和普及。

2023：思维链与 RAG

思维链（CoT）技术进一步成熟，让模型能够处理复杂的推理任务。与此同时，检索增强生成（RAG）技术兴起，解决了大模型知识截止和幻觉问题——模型可以“翻书”回答问题，而不是仅凭记忆。这一年，工具使用（Tool Use）的概念也被引入，模型开始能够调用外部 API。

2024–2025：智能体时代

Agent（智能体）成为最热门的方向。模型不再只是回答问题，而是能够自主规划、使用工具、执行代码、操作文件，完成复杂的实际任务。Claude Codex、OpenAI Codex 等编码智能体相继出现，AI 从”对话助手”进化为”行动执行者”。

技术演进的本质

回顾这条时间线，我们可以看到一条清晰的演进逻辑：

阶段	核心问题	解决方案
提示词工程	如何问对问题	Zero-shot / Few-shot / CoT
RAG	如何获取最新知识	检索 + 生成
工具使用	如何与外部交互	Function Calling
智能体	如何自主完成任务	规划 + 工具 + 记忆

本质上，这是一个让大模型从”能说”到”能做”的过程。提示词工程解决了”怎么问”的问题，RAG 解决了”知道什么”的问题，工具使用解决了”能做什么”的问题，而智能体将这些能力整合在一起，让 AI 真正成为一个能够独立完成任务的助手。

接下来，我们将逐一深入这些技术，理解它们的原理和实现。

第八章 提示词工程

8.1 什么是提示词工程

在前面的章节中，我们学习了从感知机到 Transformer 的深度学习技术。当我们训练好一个大型语言模型（如 GPT、Claude 等）后，如何使用它就成为了一个关键问题。提示词工程（Prompt Engineering）就是研究如何设计和优化输入提示（Prompt），以引导模型生成更准确、更有用的输出。

打个比方，如果把大语言模型比作一个知识渊博的专家，那么提示词就是你向这位专家提问的方式。同一个问题，不同的问法可能得到完全不同质量的答案。提示词工程就是学习如何“问对问题”的艺术。

8.2 零样本提示

零样本提示（Zero-shot Prompting）是最简单的提示方式，就是直接告诉模型你要什么，不提供任何示例。

提示：将以下英文翻译成中文：

"Hello, how are you?"

输出：你好，最近怎么样？

这种方式就像你直接问一个翻译：“这句话什么意思？”模型会基于它预训练时学到的知识直接给出答案。

8.3 少样本提示

少样本提示（Few-shot Prompting）是在提示中提供几个示例，让模型学习你想要的格式和模式。这就像教新员工时，你会先做几个示范，然后让他们模仿。

提示：

英文：I love this movie -> 情感：正面

英文：This is terrible -> 情感：负面

英文: It's okay -> 情感: 中性

英文: What an amazing experience -> 情感:

输出: 正面

在这个例子中, 我们提供了三个示例, 模型就能理解我们想要进行情感分类任务, 并按照相同的格式输出。

少样本提示之所以有效, 是因为大型语言模型具有强大的上下文学习能力 (In-context Learning)。模型虽然没有在训练时专门学习这个任务, 但它能够根据提示中的示例, 推断出任务的模式和期望的输出格式。

8.4 思维链提示

思维链提示 (Chain-of-Thought Prompting, 简称 CoT) 是一种让模型”逐步思考”的技术。对于复杂的推理任务, 直接要求答案往往效果不好, 但如果让模型展示思考过程, 准确率会大幅提升。

这就像做数学题时, 老师会要求你写出解题步骤, 而不仅仅是答案。写出步骤不仅能帮助你理清思路, 也更容易发现错误。

提示:

问题: 小明有 5 个苹果, 他给了小红 2 个, 又买了 3 个, 现在他有几个苹果?

让我们一步一步思考。

输出:

1. 小明最初有 5 个苹果
2. 他给了小红 2 个, 剩下 $5 - 2 = 3$ 个
3. 他又买了 3 个, 所以 $3 + 3 = 6$ 个
4. 答案是: 小明现在有 6 个苹果

关键在于”让我们一步一步思考”这句话, 它引导模型展示推理过程, 而不是直接猜测答案。

8.5 提示词工程的实践技巧

8.5.1 明确具体

模糊的提示往往得到模糊的回答。好的提示应该明确说明你的需求:

差的提示: 写点关于Python的内容

好的提示: 用Python写一个函数, 输入是一个整数列表, 输出是列表中的最大值。要求使用for循环实现, 并添加类型注解和文档字符串。

8.5.2 指定格式

如果你需要特定格式的输出，明确告诉模型：

提示：列出三个Python的优点，用JSON格式输出，
每个优点包含"title"和"description"字段。

8.5.3 角色扮演

让模型扮演特定角色，可以得到更专业的回答：

提示：你是一位有10年经验的高级Python开发工程师。
请review以下代码，指出潜在的问题和改进建议：
[代码内容]

8.5.4 分解复杂任务

对于复杂的任务，将其分解为多个简单的步骤：

提示：

任务：分析一篇技术文章并生成摘要

步骤1：提取文章三个核心观点

步骤2：为每个观点写一句话总结

步骤3：将三个总结组合成一段100字以内的摘要

文章内容：[文章内容]

8.6 提示词工程的本质

提示词工程之所以重要，是因为大型语言模型本质上是一个条件概率模型。给定输入序列 $x_1, x_2, \dots, x_n$ ，模型预测下一个词 x_{n+1} 的概率分布：

$$P(x_{n+1} | x_1, x_2, \dots, x_n)$$

提示词就是这个条件序列。一个好的提示能够更好地激活模型中相关的知识，引导模型生成更符合期望的输出。这就像在图书馆找书，如果你能给图书管理员更精确的描述，他就更容易找到你想要的书。

提示词工程不需要修改模型的参数，只需要调整输入，因此它是一种非常灵活和低成本的方式来优化模型的表现。

第九章 检索增强生成 (RAG)

9.1 大语言模型的局限性

在前面的章节中，我们学习了大型语言模型（LLM）的强大能力。然而，LLM 也存在一些固有的局限性：

- **知识截止**：模型的训练数据有截止日期，无法获取最新信息
- **幻觉问题**：模型可能会生成看似合理但实际上错误的信息
- **领域知识**：对于特定领域的私有数据，模型无法访问
- **上下文长度**：模型的上下文窗口有限，无法处理超长文本

打个比方，LLM 就像一个博学的人，但他的知识停留在几年前，而且无法查阅最新的资料。如果你问他今天发生了什么，他只能凭记忆回答，可能会出错。

9.2 什么是 RAG

检索增强生成 (Retrieval-Augmented Generation, 简称 RAG) 是一种结合检索和生成的技术。它的核心思想是：在生成回答之前，先从外部知识库中检索相关信息，然后将检索到的信息作为上下文提供给 LLM，让模型基于这些真实的信息生成回答。

这就像考试时允许翻书。你不需要记住所有知识，只需要知道如何查找相关信息，然后基于这些信息回答问题。

RAG 的基本流程如下：

1. 用户提出问题
2. 系统从知识库中检索相关文档
3. 将问题和检索到的文档一起提供给 LLM
4. LLM 基于检索到的信息生成回答

9.3 文本嵌入

RAG 的第一步是将文本转换为向量表示，这个过程称为文本嵌入 (Text Embedding)。

想象你在图书馆整理书籍。你会把相似主题的书放在同一个书架上。文本嵌入就是给每本书 (文本) 一个坐标 (向量)，使得内容相似的文本在向量空间中距离较近。

数学上，嵌入模型 f 将文本 t 映射到一个 d 维向量：

$$f : t \rightarrow \mathbb{R}^d$$

例如：

文本1: "机器学习是人工智能的一个分支"

文本2: "深度学习属于机器学习领域"

文本3: "今天天气很好"

嵌入后：

向量1: [0.2, 0.8, -0.3, ...]

向量2: [0.3, 0.7, -0.2, ...] // 与向量1相似

向量3: [-0.5, 0.1, 0.9, ...] // 与前两个不相似

常用的相似度度量方法是余弦相似度：

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

余弦值越接近 1，表示两个向量越相似。

9.4 向量数据库

检索到的文档需要存储在某个地方，这就是向量数据库的作用。向量数据库专门用于存储和检索高维向量。

打个比方，传统数据库就像电话簿，你通过名字 (精确匹配) 查找电话号码。向量数据库更像是一个智能地图，你说“我想找个附近好吃的中餐馆”，它会返回地理位置相近的选项。

常见的向量数据库包括：

- FAISS (Facebook AI Similarity Search)
- Pinecone
- Milvus
- ChromaDB

向量数据库的核心功能是近似最近邻搜索 (Approximate Nearest Neighbor, ANN)。当查询向量到来时，数据库快速找到与之最相似的 k 个向量。

9.5 RAG 的工作流程

让我们通过一个完整的例子来理解 RAG 的工作流程。

假设你有一个公司内部的技术文档库，员工可以提问技术问题。

9.5.1 索引阶段

首先，需要对文档进行预处理和索引：

文档1: Python 的 GIL (全局解释器锁) 确保同一时刻只有一个线程执行 Python 字节码。

文档2: 使用 `multiprocessing` 模块可以绕过 GIL 的限制，实现真正的并行计算。

文档3: `asyncio` 是 Python 的异步编程框架，适合 I/O 密集型任务。

处理步骤：

1. **分块**：将长文档切分成较小的文本块（如 500 字一块）
2. **嵌入**：使用嵌入模型（如 OpenAI 的 `text-embedding-ada-002`）将每个文本块转换为向量
3. **存储**：将向量和对应的文本存储到向量数据库

9.5.2 查询阶段

当用户提问时：

用户问题: "如何在 Python 中实现并行计算？"

处理步骤：

1. **嵌入问题**：将用户问题转换为向量
2. **检索**：在向量数据库中搜索最相似的文本块（如 top-3）
3. **构建提示**：将问题和检索到的文本块组合成提示
4. **生成回答**：LLM 基于提供的上下文生成回答

构建的提示可能如下：

基于以下信息回答问题：

[检索到的文档块1]

使用 `multiprocessing` 模块可以绕过 GIL 的限制，

实现真正的并行计算。

[检索到的文档块2]

`asyncio` 是 Python 的异步编程框架，适合 I/O 密集型任务。

问题：如何在 Python 中实现并行计算？

回答：

LLM 会基于检索到的信息生成：

在 Python 中实现并行计算，可以使用 `multiprocessing` 模块。这个模块可以绕过 GIL（全局解释器锁）的限制，实现真正的并行计算。如果需要处理 I/O 密集型任务，可以考虑使用 `asyncio` 异步编程框架。

9.6 RAG 的优势

RAG 技术具有以下优势：

- **知识实时更新**：只需更新知识库，无需重新训练模型
- **减少幻觉**：模型基于真实文档回答，可以追溯信息来源
- **领域特定**：可以构建特定领域的知识库，提供专业回答
- **成本效益**：无需微调模型，只需维护知识库

9.7 RAG 的挑战

尽管 RAG 很强大，但也面临一些挑战：

- **检索质量**：如果检索到的文档不相关，会影响生成质量
- **分块策略**：如何合理切分文档是个技术活
- **上下文长度**：检索太多文档会超出模型的上下文窗口
- **延迟**：检索和生成两个步骤增加了响应时间

针对这些问题，研究者提出了许多改进方法，如：

- **混合检索**：结合关键词检索和向量检索
- **重排序**：对检索结果进行二次排序
- **查询改写**：优化用户查询以提高检索效果

第十章 智能体与工具使用

10.1 从对话到行动

在前面的章节中，我们学习了提示词工程和 RAG 技术。这些技术让 LLM 能够更好地理解和回答问题。但是，LLM 本身只能生成文本，无法直接与外部世界交互。

想象你是一个被关在房间里的人，你知识渊博，但你无法打电话、无法上网、无法操作电脑。你只能通过门缝递纸条与人交流。这就是 LLM 的处境——它有强大的推理能力，但缺乏执行能力。

智能体（Agent）技术就是给 LLM 装上”手和脚”，让它不仅能思考，还能行动。

10.2 什么是智能体

智能体（Agent）是一个能够感知环境并采取行动以实现目标的系统。在 AI 领域，LLM 智能体通常包含以下组件：

- **大脑**：LLM 作为核心推理引擎
- **工具**：可以调用的外部功能（如搜索引擎、代码执行器、API 等）
- **记忆**：短期记忆（对话历史）和长期记忆（文件系统）
- **规划**：将复杂任务分解为可执行的步骤

打个比方，LLM 是一个聪明的大脑，智能体就是给这个大脑配上工具箱、记事本和计划表，让它能够完成实际任务。

10.3 Agent 循环：智能体的心跳

10.3.1 ReAct 框架

ReAct（Reasoning and Acting）是最经典的智能体框架，它将推理和行动结合起来。ReAct 的核心思想是让 LLM 交替进行”思考”（Reasoning）和”行动”（Acting），形成一个 Thought-Action-Observation 的循环：

用户：帮我查一下特斯拉当前的股价，并分析最近走势

Thought 1：我需要先获取特斯拉的当前股价信息

Action 1：调用股票查询工具，查询 TSLA

Observation 1：特斯拉当前股价 245.50 美元

Thought 2：现在我需要获取最近的历史数据来分析走势

Action 2：调用历史数据工具，查询 TSLA 近30天数据

Observation 2：[返回30天的股价数据]

Thought 3：根据数据，我可以分析走势了

Final Answer：特斯拉当前股价 245.50 美元，
最近30天呈现上涨趋势，涨幅约 11.6%...

10.3.2 真实的 Agent 循环

上面的 ReAct 示例做了简化。在实际的智能体系统中（如 Claude Code），Agent 循环要复杂得多。下面是一个真实 Agent 循环的核心流程：

1. **构建系统提示词**：组装系统提示词，包含工具定义、环境信息、记忆等
2. **调用模型**：将对话历史发送给 LLM，获取模型响应
3. **解析工具调用**：检查模型输出中是否包含工具调用请求
4. **权限检查**：在执行工具前，验证用户是否授权
5. **执行工具**：调用相应的工具函数，获取结果
6. **注入结果**：将工具执行结果作为新消息加入对话历史
7. **判断终止**：如果没有工具调用，说明模型认为任务完成，退出循环；否则回到第 2 步

这个过程可以用下面的伪代码表示：

```
function agentLoop(messages, tools, systemPrompt):  
    while true:  
        // 1. 调用 LLM  
        response = callModel(messages, tools, systemPrompt)  
  
        // 2. 检查是否有工具调用  
        if response has no tool_use blocks:  
            return response // 任务完成，退出循环
```

```
// 3. 执行所有工具调用
for each tool_use in response:
    tool = findTool(tool_use.name)
    result = tool.execute(tool_use.input)
    messages.append(tool_result)

// 4. 继续循环，让模型看到工具结果
```

这个循环看起来简单，但实际实现中有许多精妙的设计。

10.3.3 流式工具执行

在 Claude Code 中，工具执行并不是等模型完全输出后才开始的。当模型的输出以流 (stream) 的形式返回时，一旦某个工具调用的 JSON 参数完整接收，系统就会立即开始执行该工具，而不需要等待模型输出完毕。

打个比方，这就像餐厅里服务员还没记完你的所有菜，就已经把先点好的菜传给厨房了。这种**流式工具执行** (Streaming Tool Execution) 大大减少了等待时间。

此外，标记为“并发安全”的工具可以并行执行。比如同时搜索多个文件，不需要等一个搜索完了再搜索下一个。

10.3.4 错误恢复

真实的 Agent 循环需要处理各种异常情况。Claude Code 中实现了多层恢复机制：

- **输出截断**：当模型输出达到 token 上限时，自动提高限制并重试（最多 3 次）
- **上下文过长**：当提示词超过模型上下文窗口时，自动压缩对话历史后重试
- **API 错误**：网络或服务端错误时，自动切换到备用模型重试

这些恢复机制对用户是透明的，确保智能体能够稳定运行。

10.3.5 步数限制与“末日循环”检测

一个容易被忽视的问题是：如果智能体陷入死循环怎么办？比如反复调用同一个工具、反复修改同一个文件，却永远无法完成任务。

OpenCode 的解决方案是设置**步数上限**（默认 25 步）。当智能体连续执行超过这个上限时，系统会强制终止循环，并向用户报告当前进度。这就像给智能体设了一个“最大工作时间”，超时就必须停下来汇报。

Gemini CLI 则实现了“**末日循环**”检测 (Doom Loop Detection)：系统会监控智能体的行为模式，如果发现它连续多次执行相同的操作（比如连续 3 次调用同一个工具、传入相同的参数），就会主动介入并提醒用户。

10.3.6 持久化输入与会话恢复

在 OpenCode 的 V2 架构中，引入了一个精巧的设计——**持久化输入** (Durable Input)。用户发送的消息不会直接交给模型，而是先存入一个数据库“收件箱”，然后在安全的边界点被“提升”到对话中。

这就像邮件系统：你发出的邮件先进入对方的收件箱，对方在方便的时候才打开阅读。这种设计的好处是，即使进程崩溃，未处理的消息也不会丢失，重启后可以继续处理。

10.4 工具系统

工具使用 (Tool Use) 是智能体的核心能力。它允许 LLM 在对话过程中调用外部工具来获取信息或执行操作。

10.4.1 工具的定义

每个工具都需要告诉 LLM 三件事：我叫什么、我能做什么、你需要给我什么参数。这些信息通过 JSON Schema 来定义：

```
{
  "name": "get_weather",
  "description": "获取指定城市的天气信息",
  "input_schema": {
    "type": "object",
    "properties": {
      "city": {
        "type": "string",
        "description": "城市名称"
      },
      "date": {
        "type": "string",
        "description": "日期，如 'today', 'tomorrow'"
      }
    },
    "required": ["city"]
  }
}
```

在 Claude Code 中，工具的定义更加丰富。一个编码智能体通常配备 60 多个工具，涵盖了文件操作、代码搜索、命令执行、网络访问等方方面面。以下是 Claude Code 中的部分核心工具：

工具名称	功能描述
Bash	执行终端命令
Read	读取文件内容
Write	创建或覆盖文件
Edit	精确替换文件中的文本
Glob	按模式搜索文件名
Grep	按内容搜索文件
Agent	启动子智能体
WebFetch	获取网页内容
WebSearch	搜索引擎搜索
LSP	代码智能（跳转定义等）

表 10.1: Claude Code 核心工具列表

10.4.2 工具调用的完整生命周期

一个工具调用从开始到结束，要经过以下步骤：

- 1. **模型决策**：LLM 在输出中生成一个 `tool_use` 块，包含工具名和参数
- 2. **工具查找**：系统根据名称在已注册的工具列表中查找对应工具
- 3. **输入验证**：用 JSON Schema 验证模型传入的参数是否合法
- 4. **权限检查**：检查用户是否授权该操作（通过 Hook 系统或用户确认）
- 5. **执行工具**：调用工具的 `execute` 方法，获取结果
- 6. **结果格式化**：将结果转换为 `tool_result` 格式
- 7. **注入对话**：将结果作为用户消息加入对话历史，供模型下次读取

用一个具体的例子来说明。当用户说”帮我看看 `main.py` 的内容”时：

```
// 第1步：模型输出 tool_use 块
{
  "type": "tool_use",
  "id": "toolu_01ABC",
  "name": "Read",
  "input": { "file_path": "/home/user/main.py" }
}

// 第2-5步：系统查找 Read 工具，验证参数，执行读取
```

```
// 读取结果：
"def main():
    print('Hello, World!')

if __name__ == '__main__':
    main()"

// 第6-7步：格式化并注入对话历史
{
  "type": "tool_result",
  "tool_use_id": "toolu_01ABC",
  "content": "def main():\n    print('Hello, World!')\n..."
}
```

模型看到这个结果后，就可以基于文件内容回答用户的问题了。

10.4.3 工具执行策略

不同的智能体系统在工具执行策略上有各自的设计。

并发控制 在 Claude Code 中，标记为“并发安全”的工具可以并行执行。而 Gemini CLI 采用了一种更灵活的方式——它在每个工具的参数中自动添加一个 `wait_for_previous` 参数，让模型自己决定哪些工具可以并行、哪些必须串行。

```
// Gemini CLI 自动为工具添加的参数
{
  "name": "read_file",
  "parameters": {
    "path": {"type": "string"},
    "wait_for_previous": {
      "type": "boolean",
      "description": "是否等待前一个工具执行完毕"
    }
  }
}
```

这种设计的巧妙之处在于：把并发控制的决策权交给了模型。模型可以根据任务依赖关系，智能地决定执行顺序。

后台执行 Gemini CLI 的 Shell 工具支持后台执行。当命令耗时较长时（比如编译大型项目），智能体可以让命令在后台运行，同时继续处理其他任务。系统通过 PID 追踪来管理后台进程，确保能够获取最终结果。

```
// 后台执行示例
{
  "command": "make build",
  "is_background": true
}
// 系统返回：进程已在后台启动，PID: 12345
```

工具输出管理 工具的输出可能非常长（比如读取一个大文件）。OpenCode 实现了**有界输出机制**：工具结果被分为“预览”和“完整内容”两部分。预览保留在对话历史中供模型参考，完整内容则写入临时文件。如果模型需要完整内容，可以通过专门的工具读取。

这就像看报告时，摘要直接附在邮件正文里，完整报告作为附件。收件人先看摘要，需要时再打开附件。

10.4.4 权限与安全

智能体拥有执行能力后，安全性就变得至关重要。不同的智能体系统实现了不同层次的权限控制。

审批模式 Gemini CLI 定义了四种审批模式，从严格到宽松：

- **DEFAULT**：每次工具调用都需要用户确认
- **AUTO_EDIT**：自动批准文件编辑操作，其他操作仍需确认
- **PLAN**：计划模式，智能体只能读取和分析，不能修改文件
- **YOLO**：自动批准所有操作（适合完全信任的场景）

这种分级设计让用户可以根据任务的危险程度选择合适的模式。比如日常编码可以用 AUTO_EDIT，而处理敏感数据时用 DEFAULT。

IAM 风格的权限规则 OpenCode 采用了类似云服务的 IAM (Identity and Access Management) 权限模型。每个智能体都有自己的权限规则集：

```
// OpenCode 的权限规则示例
{
  "effect": "deny",
  "action": "bash",
```

```
"resource": "rm -rf *"
}
```

这种规则可以在智能体启动时就过滤掉不允许的工具，而不是等到执行时才发现权限不足。这就像门禁系统：不给你某间房的门禁卡，你就根本进不去，而不是进去了才发现门被锁了。

平台特定的沙箱 Gemini CLI 实现了平台特定的沙箱系统，针对 Linux、macOS、Windows 分别实现了不同的安全隔离机制。这包括文件系统访问限制、网络访问控制、进程隔离等。

主动权限建议 一个有趣的创新是 Gemini CLI 的**主动权限建议**功能。当智能体检测到某个操作可能需要特定权限时，它会主动建议用户授权，而不是等到操作失败后才提示。这提升了用户体验，减少了中断。

10.5 函数调用：连接模型与工具的桥梁

10.5.1 函数调用的工作原理

函数调用（Function Calling）是工具使用的底层实现机制。我们在第 9 章学习了 Transformer 的解码过程——模型逐词生成文本。函数调用在此基础上做了扩展：模型的输出不再仅仅是文本，还可以是结构化的工具调用指令。

具体来说，当我们将工具定义发送给 API 时，模型会在词表中增加一些特殊的 token，如 `tool_use` 标记。当模型决定调用工具时，它会生成这些特殊 token，而不是普通的文字。

```
// 发送给 API 的请求
{
  "model": "claude-sonnet-4-20250514",
  "messages": [{"role": "user", "content": "计算 123 * 456"}],
  "tools": [{
    "name": "calculate",
    "description": "执行数学计算",
    "input_schema": {
      "type": "object",
      "properties": {
        "expression": {"type": "string"}
      },
      "required": ["expression"]
    }
  ]
}
```

```
// API 返回的响应
{
  "content": [
    {"type": "text", "text": "让我帮你计算一下。"},
    {"type": "tool_use", "id": "toolu_01ABC",
     "name": "calculate",
     "input": {"expression": "123 * 456"}}
  ]
}
```

10.5.2 多轮工具调用

一个强大的智能体往往需要连续调用多个工具。每次工具调用的结果会被加入对话历史，模型基于更新后的历史继续推理，形成多轮交互：

用户：帮我创建一个 Python 项目

第1轮 - 模型调用 Bash 工具：mkdir my_project

第1轮 - 工具返回：创建成功

第2轮 - 模型调用 Write 工具：写入 main.py

第2轮 - 工具返回：写入成功

第3轮 - 模型调用 Write 工具：写入 requirements.txt

第3轮 - 工具返回：写入成功

第4轮 - 模型调用 Bash 工具：python main.py 验证

第4轮 - 工具返回：Hello, World!

第5轮 - 模型输出文本：项目已创建完成，包含以下文件...
(没有工具调用，循环结束)

10.6 系统提示词：智能体的灵魂

10.6.1 系统提示词的作用

如果说工具是智能体的”手和脚”，那么系统提示词（System Prompt）就是智能体的”灵魂”。它定义了智能体的身份、行为准则和工作方式。

系统提示词与普通提示词的区别在于：它在每次对话开始时就设定好了，贯穿整个对话过程，模型会始终遵循其中的指示。

10.6.2 系统提示词的构建

不同的智能体系统在系统提示词的构建上采用了不同的策略。

Claude Code：模块化组装 在 Claude Code 中，系统提示词由多个”片段”动态组装而成：

1. **介绍部分**：定义模型的身份
2. **行为准则**：规定模型应该如何行事
3. **工具说明**：描述每个工具的使用方法
4. **环境信息**：当前日期、工作目录、操作系统等
5. **项目上下文**：CLAUDE.md 中的项目约定
6. **记忆内容**：智能体记住的用户偏好和项目知识

Gemini CLI：分层记忆系统 Gemini CLI 实现了**分层记忆**（Hierarchical Memory）：

- **全局记忆**：7.gemini/GEMINI.md，适用于所有项目
- **扩展记忆**：来自 VSCode 等 IDE 的记忆
- **项目记忆**：.gemini/GEMINI.md，仅适用于当前项目

这种分层设计让智能体能够区分通用知识和项目特定知识。

OpenCode：Provider 特定的提示词 OpenCode 支持多个 LLM 提供商（Anthropic、OpenAI、Gemini 等），不同模型的”性格”和能力不同，因此 OpenCode 为每个 Provider 准备了不同的系统提示词模板：

```
function provider(model) {  
  if (model.includes("gpt-4")) return PROMPT_GPT4  
  if (model.includes("gemini")) return PROMPT_GEMINI  
  if (model.includes("claude")) return PROMPT_CLAUDE  
  return PROMPT_DEFAULT  
}
```

这种设计的理由是：不同模型对指令的理解方式不同，针对性的提示词能更好地发挥每个模型的优势。

上下文代数 OpenCode 的 V2 架构引入了**系统上下文代数** (System Context Algebra)。系统提示词不再是一个字符串，而是由多个”上下文源” (Context Source) 组合而成的代数结构。每个上下文源可以独立更新，系统会自动计算最终的提示词。

这就像化学反应：不同的”元素” (上下文源) 按照一定的”化学方程式” (组合规则) 反应生成最终的”化合物” (系统提示词)。当某个元素变化时，只需要重新计算，而不需要从头开始。

10.7 子智能体：分工协作

10.7.1 为什么需要子智能体

复杂的任务往往超出单个智能体的能力范围。就像一个公司需要不同部门协作一样，智能体也需要”分身”来并行处理不同的子任务。

例如，当用户说”帮我重构认证模块并更新测试”时，主智能体可以：

- 启动一个子智能体去分析认证模块的代码结构
- 同时启动另一个子智能体去查找相关的测试文件
- 收集两个子智能体的结果后，再制定重构方案

10.7.2 子智能体的隔离

子智能体在独立的上下文中运行，这就像一个员工被派到一间独立的办公室工作——他有自己的工作台 (消息历史)，看不到其他员工的工作，但他继承了公司的规章制度 (系统提示词)。

在 Claude Code 中，子智能体的上下文隔离通过以下机制实现：

- **独立的消息历史**：子智能体有自己的对话记录，不会污染主对话
- **独立的文件缓存**：子智能体读取文件时建立自己的缓存
- **关联的中止控制**：主智能体中止时，子智能体也会自动中止
- **独立的 Agent ID**：每个子智能体有唯一标识，便于追踪

10.7.3 多智能体协调模式

在更复杂的场景下，Claude Code 支持”协调者-工人” (Coordinator-Worker) 模式：

- **协调者 (Coordinator)**：负责理解任务、分配工作、综合结果。它不直接执行具体操作，而是通过派遣工人来完成任务
- **工人 (Worker)**：接收具体任务，拥有完整的工具集，独立执行任务后将结果报告给协调者

这就像一个项目经理和开发团队的关系：项目经理负责需求分析和任务分配，开发人员负责具体实现，完成后向项目经理汇报。

10.7.4 Agent 间通信协议

当多个智能体需要协作时，它们之间如何通信是一个关键问题。Google 提出了 **A2A** (Agent-to-Agent) 协议，用于规范智能体之间的通信。

在 Gemini CLI 中，子智能体有一个重要的限制：**不允许递归调用**。也就是说，子智能体不能再启动子智能体。这是为了防止无限嵌套导致的资源失控。

主智能体

子智能体 A (分析代码结构)

不能再启动子智能体

子智能体 B (查找测试文件)

不能再启动子智能体

汇总结果

这种设计虽然简单，但有效地避免了“俄罗斯套娃”式的无限嵌套问题。

10.8 上下文管理

上下文管理是智能体系统中一个容易被低估的难题。它不仅关乎对话历史的存储，更影响智能体的推理质量和成本。

10.8.1 上下文压缩

当对话历史接近模型的上下文窗口上限时，所有智能体系统都会进行压缩。但压缩的策略各有不同。

Claude Code 的压缩相对直接：将早期消息总结为摘要，保留最近的对话。

OpenCode 的压缩更加结构化，它会生成一个包含以下字段的摘要：

- **目标**：用户想要完成什么
- **约束**：有哪些限制条件
- **进展**：已完成、进行中、被阻塞的任务
- **关键决策**：做出了哪些重要决定
- **下一步**：接下来应该做什么
- **相关文件**：涉及哪些文件

这种结构化的压缩能更好地保留关键信息，避免智能体在压缩后“忘记”重要上下文。

10.8.2 历史加固

Gemini CLI 实现了一个独特的功能——**历史加固** (History Hardening)。在将对话历史发送给模型之前，系统会对历史进行“清洗”，移除或修正可能误导模型的内容。

打个比方，这就像在把会议纪要发给参会者之前，先由助理检查一遍，修正其中的错别字和歧义表述，确保每个人看到的都是清晰准确的信息。

10.8.3 上下文纪元

OpenCode V2 引入了**上下文纪元** (Context Epoch) 的概念。一个纪元是指系统提示词保持不变的一段时间跨度。当发生以下事件时，会开启新的纪元：

- 上下文压缩
- 切换模型或提供商
- 切换智能体

每个纪元都有一个“基线” (Baseline)，记录了该纪元开始时的系统上下文状态。这就像版本控制系统中的“快照”——你可以随时回到某个快照的状态。

这种设计的好处是：在一个纪元内，系统提示词是不可变的，这保证了模型推理的一致性。当需要变更时，通过纪元切换来管理，而不是在对话中途偷偷修改提示词。

10.8.4 图结构上下文

Gemini CLI 采用了最复杂的上下文管理方案——**图结构上下文** (Graph-based Context)。传统的上下文管理是一个线性的消息列表，而 Gemini CLI 将上下文表示为一个有向图：

- **节点** (Node)：每条消息是一个节点
- **处理器** (Processor)：对节点进行变换的管道
- **工作缓冲区** (Working Buffer)：处理器可以修改的可变状态

这种设计允许对上下文进行复杂的变换操作，比如：重新排序消息、注入额外的上下文、过滤不相关的内容等。

打个比方，线性上下文就像一条流水线，原料进去，产品出来。而图结构上下文更像一个化工厂，原料可以在不同的反应罐之间流转，经过各种处理，最终生成产品。

10.9 记忆系统

10.9.1 短期记忆：对话历史

短期记忆就是当前对话的消息列表。它让智能体能够理解对话的连贯性。

用户：帮我查一下北京的天气

智能体：[调用天气工具] 北京今天晴，22°C

用户：那上海呢？ // 智能体从上下文知道还是在问天气

智能体：[调用天气工具] 上海今天多云，25°C

短期记忆面临的一个挑战是**上下文窗口限制**。模型的输入长度是有上限的，当对话很长时，早期的内容会被“遗忘”。为了解决这个问题，Claude Code 实现了**自动压缩**机制：当对话历史接近上下文窗口上限时，系统会自动将早期的消息压缩成摘要，保留关键信息，释放空间给新的内容。

10.9.2 长期记忆：文件系统

长期记忆让智能体能够跨会话记住信息。在 Claude Code 中，长期记忆通过文件系统实现。

具体来说，智能体会在项目目录下维护一个 `memory/` 文件夹，其中 `MEMORY.md` 是记忆的索引文件。智能体在工作过程中学到的重要信息（如用户偏好、项目约定、常见问题解决方案）会被写入这些文件。

下次启动时，智能体会自动读取这些记忆文件，恢复之前的“记忆”。

// 记忆文件示例（`~/.claude/projects/xxx/memory/MEMORY.md`）

项目记忆

用户偏好

- 使用 TypeScript 而非 JavaScript
- 偏好函数式编程风格
- 测试框架使用 vitest

项目约定

- API 路由放在 `src/routes/` 目录
- 数据库操作使用 Prisma ORM
- 提交信息遵循 Conventional Commits

这种设计非常巧妙：它不需要修改模型参数，仅仅通过文件系统就实现了持久化记忆。就像一个工程师把自己的工作笔记放在抽屉里，下次上班时翻开笔记本就能回忆起之前的工作。

10.10 编码智能体：Claude Code

10.10.1 编码智能体的特点

编码智能体（Coding Agent）是专门用于编程任务的智能体。相比通用智能体，编码智能体需要更强大的代码理解和操作能力。

Claude Code 是 Anthropic 推出的编码智能体。它基于 Claude 模型，配备了丰富的开发工具，能够理解整个代码库的上下文，自主完成代码编写、调试、重构等复杂任务。

10.10.2 一个完整的工作流程

让我们通过一个完整的例子来理解 Claude Code 的工作流程。假设用户说：“帮我修复登录页面的 bug，点击按钮没有反应”。

Claude Code 的执行过程如下：

[思考] 用户报告登录按钮点击无反应，我需要先找到相关代码

[行动] 调用 `Grep` 工具，搜索 "login" 相关文件

[观察] 找到 `src/pages/Login.tsx`

[思考] 需要查看登录页面的代码

[行动] 调用 `Read` 工具，读取 `Login.tsx`

[观察] 发现 `handleSubmit` 函数中有一个拼写错误

[思考] 需要确认这个函数的调用方式

[行动] 调用 `LSP` 工具，查找 `handleSubmit` 的引用

[观察] 确认是按钮的 `onClick` 事件处理函数

[思考] 问题已定位，现在修复这个拼写错误

[行动] 调用 `Edit` 工具，将 `"formData"` 改为 `"formdata"`

[观察] 修改成功

[思考] 需要验证修复是否正确

[行动] 调用 `Bash` 工具，运行测试

[观察] 所有测试通过

[最终回答] 问题已修复。原因是 `handleSubmit` 函数中

`"formData"` 被拼写为 `"fromData"`，导致获取不到表单数据。

已修改 `src/pages/Login.tsx` 第42行，测试全部通过。

在这个过程中，Claude Code 自主完成了“搜索-阅读-分析-修复-验证”的完整流程，总共调用了 5 次工具，经历了 5 轮思考-行动循环。

10.10.3 技能系统

Claude Code 还实现了**技能**（Skills）系统，可以看作是针对特定任务的“专家模式”。每个技能定义了一组专用的提示词和工具集，当触发某个技能时，智能体会切换到该技能的工作方式。

例如，一个代码审查技能可能包含：

- 专门的审查提示词（关注安全漏洞、性能问题等）

- 限制只使用只读工具（Read、Grep、Glob），不允许修改文件
- 输出格式要求（按严重程度分类列出问题）

技能可以通过斜杠命令（如 `/review`）手动触发，也可以由智能体根据上下文自动选择。

10.11 智能体的挑战与展望

10.11.1 当前挑战

尽管智能体技术发展迅速，但仍面临一些挑战：

- **可靠性**：智能体可能会做出错误的决策或调用错误的工具，特别是在复杂的多步任务中，一个错误可能导致后续步骤全部失败
- **安全性**：需要防止智能体执行危险操作，如删除重要文件、泄露敏感信息等
- **效率**：多步推理和工具调用会增加响应时间和 API 成本
- **可控性**：用户希望了解智能体在做什么，并能在必要时介入

10.11.2 未来展望

智能体技术正在快速发展，未来的趋势包括：

- **多智能体协作**：多个专业化的智能体协同工作，就像一个高效的开发团队
- **自主学习**：智能体能够从经验中学习，不断改进策略
- **更丰富的工具生态**：MCP（Model Context Protocol）等标准让工具的接入更加规范化和便捷
- **个性化**：智能体能够适应不同用户的偏好和工作方式，通过记忆系统越用越懂你

智能体代表了 AI 从“能说”到“能做”的重要跨越。从提示词工程解决“怎么问”，到 RAG 解决“知道什么”，到工具使用解决“能做什么”，再到智能体将这些能力整合在一起——我们正在见证 AI 从“对话助手”进化为“行动执行者”的历史性转变。